

SuppliWise

System Manual and Technical Reference

An AI-assisted personalised dietary supplement advisory platform with subscription entitlements and a tamper-evident provability layer

Institution	[Institution / University Name]
Course / Program	[Course Code and Title]
Section	[Section]
Capstone Project Title	SuppliWise: AI-Powered Personalized Supplement Advisory System with On-Chain Provability
Group Members	[Member 1 · Member 2 · Member 3 · Member 4]
Instructor / Adviser	[Instructor Name] · [Adviser / Panel Adviser Name]
Academic Year	[A.Y. 2026–2027, Second Semester]
Document Version	1.0 — [Date of Submission]
Repository	github.com/REIldes/SuppliWise

This document is the consolidated engineering reference for the SuppliWise capstone project. It is generated from the source tree and is intended as the technical baseline for the team's written deliverables. Every figure in this manual is reproducible from the repository.

Table of Contents

Purpose Consolidated technical reference for all capstone written deliverables.
Scope Full system: frontend, backend, AI layer, provability layer, security, testing, deployment.
Source of truth Generated from the repository source tree; regenerate with `python docs/generate_system_manual.py`
Status v1.0 — [Date of Submission]

Chapters

1 Introduction

- 1.1 Background and Problem Statement
- 1.2 Objectives of the Study
- 1.3 Scope and Delimitation
- 1.4 Significance of the Project

2 System Overview

- 2.1 Product Summary
- 2.2 Actors and User Roles
- 2.3 Functional Capability Map
- 2.4 Codebase Metrics

3 Requirements Analysis

- 3.1 Functional Requirements
- 3.2 Non-Functional Requirements
- 3.3 Requirements Traceability Matrix

4 System Architecture

- 4.1 Architectural Style
- 4.2 Logical Layer Diagram
- 4.3 Technology Stack
- 4.4 Repository Layout
- 4.5 Request Lifecycle and Middleware Pipeline
- 4.6 Core Data Flows
- 4.7 Key Design Decisions

5 Frontend Subsystem

- 5.1 Build Toolchain and Configuration
- 5.2 Routing and Route Guards
- 5.3 Page Inventory
- 5.4 Component Inventory
- 5.5 Client State and Session Model
- 5.6 PWA and Android Packaging

6 Backend Subsystem

- 6.1 Bootstrap and Boot-Time Checks
- 6.2 Route Groups and API Surface
- 6.3 Data Model
- 6.4 Utility and Service Layer

7 Artificial Intelligence Subsystem

- 7.1 Purpose and Design Constraints
- 7.2 Provider Routing Table
- 7.3 Recommendation Pipeline
- 7.4 Priority Assessment Flagging
- 7.5 Chat Assistant and Safety Guardrails
- 7.6 Deterministic Clinical Fallback
- 7.7 AI Privacy Controls

8 Subscription and Entitlements

- 8.1 Subscription Tiers
- 8.2 Billing Model
- 8.3 Purchase and Approval Workflow
- 8.4 Entitlement Enforcement
- 8.5 Currency Resolution

9 Provability Layer (Web3)

- 9.1 Positioning and Scope
- 9.2 Chain Architecture
- 9.3 Feature Inventory
- 9.4 Foundation Guarantees
- 9.5 Public Attack Surface

10 Security Architecture

- 10.1 Defence Model
- 10.2 Authentication and Multi-Factor Authentication
- 10.3 Session Architecture
- 10.4 Authorization Model
- 10.5 Rate Limiting and Body Protection
- 10.6 Secret Custody
- 10.7 Security Event Log
- 10.8 Live Security Monitor and Audit

11 Testing and Quality Assurance

- 11.1 Test Strategy
- 11.2 Test Inventory
- 11.3 Continuous Integration Pipeline
- 11.4 Coverage Matrix

12 Deployment and Operations

- 12.1 Prerequisites
- 12.2 Local Development Setup
- 12.3 Environment Configuration
- 12.4 Production Readiness
- 12.5 Android APK Build
- 12.6 Operational Tooling

13 Limitations and Future Work

- 13.1 Declared Limitations
- 13.2 Future Enhancements

14 Team and Contribution Record

- 14.1 Team Roster
- 14.2 Module Ownership

A Appendix A — API Endpoint Reference

- A.1 Authentication — `/api/auth``
- A.2 Account Security
- A.3 Member Domain
- A.4 Subscription
- A.5 Administrator
- A.6 Provability (`/api/web3``)

B Appendix B — Database Collection Reference

- B Collections and Indexes

C Appendix C — Environment Variable Reference

- C Configuration Variables

D Appendix D — Source File Index

- D File-by-File Index

E Appendix E — Glossary

- E Terms and Abbreviations

Chapter 1 · Introduction

1.1 Background and Problem Statement

Dietary supplement use has grown substantially, yet most consumers begin a regimen with little more than advertising-derived information. Product labels, influencer recommendations and generic listicles do not account for an individual's age, medical conditions, current medications, allergies, diet pattern, sleep quality or lifestyle exposures. The result is a recognisable and well-documented failure mode: people take supplements they do not need, at doses they cannot tolerate, or that interact with prescription medication they are already on.

Generic recommendation systems widen this gap rather than close it. A tool that asks for age and weight and then returns the same list to every user with that combination is not a health instrument; it is a catalogue with extra steps. Meaningful personalisation requires a structured intake that captures clinical context, a reasoning layer that can interpret that context, and a presentation layer honest enough to say what the evidence does and does not support.

A second problem compounds the first: **trust**. A recommendation a user cannot trace is a recommendation a clinician cannot use. Personalised wellness output is typically unverifiable — the inputs are private, the model is opaque, and the supplement supply chain is invisible. SuppliWise was designed against both problems simultaneously: personalised output is **explainable at the point of recommendation** (each card states the exact reported condition that triggered it), and a **tamper-evident provability layer** records what happened, when, and on what inputs, without ever writing personal health data outside the database.

POSITIONING SuppliWise is an **advisory tool, not a medical device**. Every recommendation uses possibility language ("may support", "evidence suggests"), every generated report carries a consult-a-professional notice, and the system never issues a diagnosis. This constraint is enforced in the prompt rules, in the output sanitiser, and in the generated PDF.

1.2 Objectives of the Study

- The project set out to build and demonstrate a working personalised supplement advisory system. The specific objectives are:
1. Design and implement a structured multi-step health assessment that captures the clinical variables relevant to supplement safety and necessity.
 2. Integrate a large language model behind a **provider-routing abstraction** so no single vendor outage or credit exhaustion degrades the product.
 3. Guarantee that the system always produces a result, by implementing a deterministic rule-based clinical engine that runs whenever the AI path is unavailable.
 4. Make every recommendation traceable to the user's own reported data, and attach peer-reviewed literature citations to the generated plan.
 5. Implement adherence tracking — daily intake, streaks, wellness scoring and adherence percentage — so the system measures behaviour, not just advice.
 6. Implement a four-tier commercial model with **server-side** entitlement enforcement and a human-reviewed purchase approval workflow.
 7. Add an authentication and account-security layer meeting enterprise expectations: passkeys, TOTP, email OTP, recovery codes, step-up re-authentication and device session management.
 8. Add a tamper-evident audit layer that anchors sensitive events to an append-only proof-of-work ledger without exposing personal health data.
 9. Build a complete administrator control panel with user management, subscription review, support inbox and a live security monitor.
 10. Package the system as an installable Progressive Web App and a native Android application.
 11. Sustain a defensible engineering practice through an automated test suite (105 test files) and a CI pipeline that gates syntax, tests, builds and dependency advisories.

1.3 Scope and Delimitation

Dimension	In scope	Out of scope / Delimitation
Domain	Dietary supplement advisory, adherence tracking, subscription entitlements, product provability	Clinical diagnosis, prescription, drug dispensing, medical-record keeping
Users	Members (end users), Administrators (project staff)	Third-party clinicians beyond the read-only shared-profile view
Platform	Web (React PWA), Android (Capacitor APK), Node.js backend, MongoDB database	Native iOS build, desktop installer, other databases
AI usage	Recommendation synthesis, supplement deep-dive, chat assistant, severity second-opinion, system threat prediction	Fine-tuning or training a proprietary model; autonomous clinical decisions
Blockchain	In-app append-only proof-of-work ledger used as a tamper-evident audit anchor, with content addressing and tokenised rewards	External public chain, real cryptocurrency, gas fees, consensus across nodes, peer-to-peer value transfer on a public network
Payments	Manual proof-of-payment submission with administrator review and server-computed pricing	Integrated payment gateway or automatic settlement
Records	Assessment history retained 5 calendar years; account and audit records retained while the account exists	Long-term medical record archiving, statutory health-record compliance

1.4 Significance of the Project

The academic value of the project lies in three deliberate engineering positions, each of which required a defensible argument rather than a default choice:

- ****Airlock architecture for AI.**** AI is treated as an unreliable, external, networked dependency. Every AI call sits behind a router with a declared provider, a declared model and a health probe; the product has a deterministic path that requires no network at all.
- ****Server-authoritative entitlements.**** A payroll enforced in the client is not a payroll. All feature gating resolves on the server, and the frontend mirror exists only to render a lock state instantly.
- ****Provability without exposure.**** The audit requirement and the privacy requirement usually conflict. The project resolves it by reducing anything sensitive to a SHA-256 digest over canonical JSON before it is ever anchored.

Chapter 2 · System Overview

2.1 Product Summary

SuppliWise is a full-stack web application that converts a structured health assessment into a personalised, evidence-cited supplement plan, then measures whether the user actually follows it. The member journey is: register → complete a four-step assessment → receive an AI-generated plan → add supplements to a daily plan → mark doses taken → watch adherence, streaks and a wellness score build → export the whole thing as a formatted clinical PDF.

Around that core sit three subsystems that a capstone panel reliably probes: a four-tier **subscription model** with server-enforced entitlements; a **support and notification** layer that separates an AI assistant from a real human conversation; and a **provability layer** that anchors supply-chain, escrow, rewards and consent events to an in-app ledger.

A user signs up for supplement advice. The engineering problem underneath is a multi-provider AI pipeline with a guaranteed-offline fallback, a paywall that cannot be bypassed from the browser, and an audit trail that is provably tamper-evident without ever exposing a health record.

2.2 Actors and User Roles

Actor	Identity Space	Capabilities	Entry Point
Guest	Public routes	View the marketing landing page and the public pricing catalogue; obtain a server-issued verification or share code	/, /pricing, /verify/:code, /share/:token
Member	Session-scoped user token	Complete assessments, receive and manage recommendations, track intake, view insights and history, export PDFs, manage profile and account security, open support threads, use entitled AI and Web3 features	/login, /signup → /dashboard
Administrator	Separate admin token space, /admin	Review and decide purchase and cancellation requests; manage user accounts, subscription entitlements, admins and assessments; operate the support inbox; read the live security monitor and the audit record; issue credential notices	/admin/login (alias + password + TOTP)
Clinician (external)	No account	Read a time-boxed, revocable, read-only shared health profile link	/share/:token
Verifier (external)	No account	Recompute every anchored proof for a supply-chain batch via QR scan or code entry	/verify/:code

NOTE The administrator identity space is deliberately **separate from the member app**. Admin routes are guarded by `protect + adminOnly`, a user token is refused on them, and an admin token is refused on member routes. No navigation path connects the two, and the frontend session store is keyed by token so an in-place account switch cannot hand one identity's plan to the next.

2.3 Functional Capability Map

Subsystem	Capability	Key Route
Assessment	Four-step wizard: demographics → diet and goals → medical conditions and symptoms → lifestyle, medications, allergies, blood tests. Session-persisted progress, unit conversion, spam detection, age-aware tooltips.	POST /api/assessment
AI Plan Generation	Structured wellness plan with up to 20 prioritised supplement recommendations, each carrying a priority band, confidence score, trigger reasons, condition context, dosage, timing, interactions and citations.	POST /api/recommend
Adherence Tracking	Daily intake plan grouped into time slots, mark taken/undo, bulk tick, streaks, wellness score, weekly adherence, calendar history.	GET/POST /api/dashboard
Insights	Today's progress, weekly adherence chart, wellness phase, lifestyle recommendations, AI insight panel.	GET /api/insights
Assessment History	Paginated archive of past assessments with full stored AI results, view, PDF export and delete.	GET /api/assessment/history
Document Export	Formatted A4 PDF clinical report generated client-side with jsPDF, plus a separate administrator security report.	Client-side (jsPDF)
AI Chat Assistant	Floating assistant scoped to health and SuppliWise topics, multi-turn, typo-tolerant, plan-gated server-side.	POST /api/chat
Support	Stored human conversation threads with category, status, unread state and a two-pane administrator inbox.	/api/support-chat, /api/admin/chats
Notifications	In-app inbox with unread count, mark-read, mark-all-read, delete-one and delete-all-read.	/api/notifications
Subscriptions	Public plan catalogue, purchase requests with proof of payment, administrator approval queue, cancellations, live plan-change stream.	/api/subscription
Account Security	Password policy and change, passkeys, TOTP, email OTP, recovery codes, recovery email, step-up re-authentication, device list and revocation, security event history.	/api/security, /api/auth/*

Subsystem	Capability	Key Route
Provability Layer	Twenty features covering supply chain, certification, escrow, marketplace, identity, health ledger, data sovereignty, storage, rewards, NFTs, staking, loyalty, governance, knowledge, disputes, profile sharing, verifiable AI, trial consent, oracle feeds and expert consultation.	/api/web3
Admin Console	Eight tabs: Overview, Users, Admins, Subscriptions, Chats, Assessment, AI and Security — with a live 45-probe security monitor.	/api/admin

2.4 Codebase Metrics

The following figures are measured from the repository at the time of writing and are the numbers to quote in the written deliverables.

Metric	Value	Notes
REST endpoints	220	Across 22 server route files plus 9 Web3 sub-routers
Server source	138 files / ~41,760 lines	JavaScript, excluding tests
Server test suite	63 files / ~20,677 lines	node --test; ~658 tests (619 executed, 39 environment-gated skips)
Frontend source	206 files / ~89,906 lines	JSX, JS and CSS under src/
Frontend source (no tests)	164 files / ~80,872 lines	70 JSX files, ~35,002 lines
Frontend test suite	42 files	node --test; ~244 tests
Total test files	105	63 server + 42 frontend
Pages	25	src/Pages/*.jsx
Components	43	src/Components/**/*.jsx
Mongoose models	20 files	Producing 33 core collections plus 25 Web3 collections in a single Web3.js
Server utilities	62 files	server/utills/ — the service layer
Commits	102	2026-05-27 through 2026-10-02
Contributors	4	See Chapter 14
Documentation files	118 markdown files	At the repository root, plus the docs/ manual set

CAUTION
Report code volume as **lines of source and lines of test separately**. Combining them overstates the implementation; the test-to-source ratio is itself a stronger capstone metric (approximately 0.49 : 1 server-side and 0.11 : 1 client-side by file count).

Chapter 3 · Requirements Analysis

3.1 Functional Requirements

Functional requirements are numbered FR-n and are traceable to the implementing module in Section 3.3.

ID	Requirement	Priority
FR-01	The system shall register an account with first name, last name, gender, email, date of birth and a password that satisfies the shared password policy, validated against a server-issued CAPTCHA.	Must
FR-02	The system shall authenticate a registered member and, where a second factor is enabled, require completion of that factor before a session is issued.	Must
FR-03	The system shall authenticate an administrator using alias, password and an authenticator-app code, with a mandatory password-change gate until satisfied.	Must
FR-04	The system shall collect a four-step health assessment covering demographics, diet type, health goals, medical conditions, symptom severity, medications, allergies, lifestyle habits, blood test results and pregnancy or breastfeeding status where applicable.	Must
FR-05	The system shall preserve in-progress assessment state so the user may navigate away and resume.	Must
FR-06	The system shall generate a personalised supplement plan of up to 20 recommendations, each prioritised high, medium or low and carrying a confidence score.	Must
FR-07	Each recommendation shall state the reported conditions, symptoms or goals that triggered it, and shall include dosage, timing, interaction warnings and a professional disclaimer.	Must
FR-08	Each recommendation shall cite supporting peer-reviewed literature, including journal, authors and PMID where available.	Should
FR-09	The system shall return a valid result even when no AI provider is reachable, using a deterministic rule-based clinical engine.	Must
FR-10	The system shall let the user add and remove supplements from a daily plan and record each dose as taken or undo.	Must
FR-11	The system shall compute and display a wellness score, a daily streak and an overall adherence percentage.	Must
FR-12	The system shall display today's progress, weekly adherence and a calendar view of completion history.	Must
FR-13	The system shall retain past assessments with their stored AI results and allow the user to view, export to PDF, or delete them.	Must
FR-14	The system shall generate a formatted A4 PDF clinical report from results, history or the administrator console.	Must
FR-15	The system shall provide an AI chat assistant that answers health, nutrition and SuppliWise questions and politely declines off-topic requests.	Should
FR-16	The system shall store administrator conversations with member support threads and surface unread state to both sides.	Must
FR-17	The system shall present a public plan catalogue with prices in the user's detected currency.	Must
FR-18	The system shall allow a member to submit a plan request with a payment reference and an optional proof-of-payment image.	Must
FR-19	The system shall allow an administrator to approve or decline plan and cancellation requests, optionally overriding the number of days granted, and shall record the decision.	Must
FR-20	The system shall enforce feature entitlements on the server and reflect plan changes in the browser without a page reload.	Must
FR-21	The system shall let a member register a passkey, rename it and remove it, and shall allow passwordless sign-in with a passkey.	Should
FR-22	The system shall let a member enable an authenticator app or email codes as a second factor, and shall support single-use recovery codes.	Must
FR-23	The system shall require step-up re-authentication before any security-mutating action.	Must
FR-24	The system shall list signed-in devices and allow revoking one device, all other devices, or every session.	Must
FR-25	The system shall show the member their own security event history.	Must
FR-26	The system shall provide the administrator a live security monitor reporting per-probe status, framework, implementing file and latency.	Should
FR-27	The system shall support supply-chain batch tracking with a public QR verification view that recomputes every proof.	Should
FR-28	The system shall support escrow-funded marketplace orders that release on delivery confirmation, with a juror-based dispute process.	Should
FR-29	The system shall support a stake-weighted, quorum-gated DAO that applies an approved parameter change to live configuration.	Should
FR-30	The system shall support a time-boxed, revocable, account-free shared health profile link for clinicians.	Should
FR-31	The system shall issue idempotent WELL token rewards for check-ins, tracked intake and completed assessments.	Should
FR-32	The system shall record data-sharing consent and clinical-trial consent on the ledger, including withdrawal.	Should
FR-33	The system shall run as an installable PWA and be buildable as an Android APK with in-app PDF download.	Should

ID	Requirement	Priority
FR-34	The system shall push plan changes to open browsers over a live event stream.	Should
FR-35	The system shall refuse to start in production when security-critical configuration is absent.	Must

3.2 Non-Functional Requirements

ID	Category	Requirement	Implementation Evidence
NFR-01	Security	Passwords shall be stored with a memory-hard algorithm and a unique salt.	argon2id (19 MiB, t=2, p=1) via hash-wasm; legacy bcrypt verified and transparently upgraded
NFR-02	Security	Second-factor secrets shall be encrypted at rest.	AES-256-GCM envelope keyed from TOTP_ENCRYPTION_KEY, with key-id rotation support
NFR-03	Security	Repeated failed authentication shall be metered and escalating.	Ladder of 15 min → 1 h → 6 h → 24 h with decaying offence history; 2FA ladder capped at 1 h
NFR-04	Security	Login shall not disclose whether an account exists.	Identical response body and comparable timing; the unknown-user path burns a throwaway password compare
NFR-05	Security	HTTP responses shall carry hardened headers.	Helmet with a restrictive Content-Security-Policy; explicit CORS allowlist (never a wildcard with credentials)
NFR-06	Security	Request bodies shall be metered before buffering.	Global in-flight byte budget plus Content-Length pre-check, both mounted before express.json()
NFR-07	Security	Object reads shall be owner-scoped.	Every find pairs the object id with req.user._id; a miss returns 404, never 403
NFR-08	Security	Security-relevant activity shall be auditable.	Append-only SecurityEvent log with a closed enum of event types and a fixed metadata allowlist
NFR-09	Privacy	Personal health data shall never be written to the ledger.	Sensitive values reduced to a SHA-256 digest over canonical JSON before anchoring
NFR-10	Privacy	AI calls shall exclude identifying data.	Only health fields transmitted; names, emails, pictures, phone numbers and handles stripped before any model call
NFR-11	Reliability	The product shall function without AI availability.	Rule-based clinical fallback covers all age groups, medications, conditions, symptoms, diet, lifestyle and pregnancy
NFR-12	Reliability	Provider health shall be observable.	Per-provider live probe distinguishing Reachable, Key rejected, Model unavailable and Unreachable, with measured latency
NFR-13	Performance	The initial page payload shall be small.	Route-level code splitting; jsPDF, jspdf-autotable and the WebAuthn browser SDK lazily imported
NFR-14	Performance	API response time shall stay interactive.	Target sub-500 ms for dashboard reads, enforced by compound indexes on user, assessment and day key
NFR-15	Portability	The system shall run on desktop and mobile browsers and package as a native Android application.	Responsive single-column layouts; PWA plus Capacitor 8
NFR-16	Portability	The system shall run on Node.js without native build tooling.	argon2id via WASM (hash-wasm) instead of a native binding
NFR-17	Maintainability	Business rules shall be defined once.	Single definitions for password policy, entitlements, plan day, name validation and AI routing, with contract tests that fail on drift
NFR-18	Maintainability	Security-critical configuration shall be validated at boot.	Boot refuses to start on missing or placeholder JWT secrets, absent TOTP keys, misconfigured WebAuthn, or debug switches enabled in production
NFR-19	Testability	Critical security behaviour shall be automated.	105 test files including real-HTTP end-to-end suites and contract tests
NFR-20	Traceability	A defect must not reach a review unnoticed.	CI gates syntax, tests, lint, production build, dependency audit and a credential sweep on every push and pull request
NFR-21	Usability	The UI shall remain usable on small screens.	Single-column layouts, collapsible sections, touch-optimised controls, reduced-motion and increased-contrast support
NFR-22	Recoverability	A recoverable client failure shall not present as a blank screen.	Global error boundary with one-shot self-heal for stale dynamic-import chunks

3.3 Requirements Traceability Matrix

Requirement Group	Implementing Module	Verification
FR-01 – FR-03	routes/auth.js, utils/password.js, utils/passwordRules.js, utils/totp.js, middleware/auth.js	auth-security, password-policy-agreement, admin-session-expiry, admin-forced-password-change
FR-04 – FR-05	Pages/AssessmentPage.jsx, routes/assessment.js, models/Assessment.js	assessment-history, plan-day
FR-06 – FR-09	routes/recommend.js, utils/aiRouter.js, utils/sanitize.js	ai-routing, ai-default-provider, recommendation-plain-language
FR-10 – FR-12	routes/dashboard.js, utils/planDay.js, utils/wellnessScore.js, Pages/TrackIntakePage.jsx	wellness-missed-penalty, intake-time-slots, plan-day-routes, slotSchedule
FR-13 – FR-14	Pages/HistoryPage.jsx, utils/wellnessReport.js, utils/exportPDF.js	vite build, manual export review
FR-15	routes/chat.js, utils/chatSafety.js, Pages/ChatAssistant.jsx	chat-safety
FR-16	routes/supportChat.js, routes/adminSupportChats.js, Components/SupportInbox, Components/AdminSupportChats	support-chat-state, support-chat.e2e
FR-17 – FR-19	utils/planCatalogue.js, utils/subscriptionState.js, utils/subscriptionRequests.js, routes/admin.js	plan-catalogue, subscription-state, subscription-requests, subscription-request-routes
FR-20	utils/entitlements.js, routes/subscription.js, hooks/useSubscription.js, utils/subscriptionBus.js	subscription-entitlements, features (frontend mirror)
FR-21 – FR-25	routes/passkeys.js, routes/totp.js, routes/recoveryCodes.js, middleware/stepUp.js, routes/security.js	passkeys, totp-and-recovery, totp-encryption, recovery-codes, recovery-code-generation, sessions-and-csrf
FR-26	routes/admin.js, utils/attack_probes.js, utils/securityAudit.js, Components/SecurityStatus	securityStatusView, live monitor polling
FR-27 – FR-32	blockchain/ledger.js, blockchain/engine.js, blockchain/rules.js, routes/web3/*	web3, web3-plan-gate, plus node test-web3-flows.js end-to-end probe
FR-33	vite.config.js, capacitor.config.json, android/app/src/main/java/com/suppliwise/app/*	vite build, APK build
FR-34	routes/subscription.js (SSE), utils/subscriptionBus.js	subscription-state
FR-35	index.js boot checks, utils/origins.js, utils/secretBox.js	env-file, env-usage, CI server job

Chapter 4 · System Architecture

4.1 Architectural Style

SuppliWise is a **three-tier, single-page-application architecture** with a layered backend. The frontend is a React SPA that owns presentation and short-lived UI state only. The backend is a stateless Express application that owns every decision of substance — authentication, authorisation, entitlement, pricing, persistence and AI orchestration. All durable state lives in MongoDB.

Three properties of the backend are architectural rather than incidental:

- **Server authority.** No security-relevant or commercial decision is taken in the browser. The client renders a state; the server decides whether it is permitted.
- **Layered utilities.** Business logic lives in `server/utils/` rather than inside route handlers, which is what makes the rules unit-testable in isolation and reusable across routes.
- **Explicit middleware ordering.** The middleware sequence in `index.js` is a security control, not a convenience. Rate limiting and body metering are mounted deliberately *before* `express.json()` so no request body is buffered until the request has been admitted.

4.2 Logical Layer Diagram

Figure 1 — SuppliWise logical architecture. Each box reads top to bottom.



4.3 Technology Stack

Layer	Technology	Version	Rationale
Frontend	React	18.2	Mature component model; lazy routes are a first-class pattern

Layer	Technology	Version	Rationale
Build	Vite	8.x	Fast dev server; route-level code splitting; separate cache directories so a build cannot invalidate a running dev server
Routing	React Router	7.x	Declarative nested routes with loader-style guards
Mobile (web)	vite-plugin-pwa / Workbox	1.3 / 7.4	Installability, offline asset caching, automatic updates
Mobile (native)	Capacitor	8.x	Wraps the built SPA in a native Android shell with a WebView download handler
Backend	Node.js + Express	≥18 / 4.x	node --test needs no framework; built-in fetch removes an HTTP client dependency
Database	MongoDB + Mongoose	8.x	Schema validation with a schemaless Mixed field for AI output, so no migration is needed when the model output shape changes
Authentication	jsonwebtoken	9.x	HS256 pinned by the verifier
Password hashing	hash-wasm (argon2id)	4.12	Memory-hard hashing with no native build step; bcrypt accepted for legacy hashes
WebAuthn	@simplewebauthn/server + /browser	14.x	Audited library instead of hand-rolled ceremony crypto
TOTP	speakeasy	2.x	Standards-compliant RFC 6238
Symmetric crypto	Node crypto (AES-256-GCM)	built-in	Envelopes TOTP seeds and provability private keys at rest
Asymmetric crypto	Node Web Crypto (ed25519)	built-in	Identity keys and transaction signatures
Transport security	helmet + cors + express-rate-limit + compression	8 / 2.8 / 7.5 / 1.8	Header hardening, origin allowlist, request metering, response compression
Email	nodemailer	10.x	SMTP transport with Gmail App Password support
QR generation	qrcode	1.5	Supply-chain verification codes
PDF	jspdf + jspdf-autotable	4.2 / 5.0	Client-side report generation, no server round-trip
Testing	node --test	built-in	Zero-dependency runner; the project's stated preference for a dependency-light toolchain
CI	GitHub Actions	—	Four jobs gating syntax, tests, builds, audits and credential leakage

4.4 Repository Layout

Figure 2 — repository tree (abridged; Appendix D is the full index)

```
SuppliWise/
|-- .github/workflows/ci.yml      # syntax, tests, builds, audit, secret sweep
|-- docs/                        # manuals, guides, this generator
|-- server/                      # ---- BACKEND ----
|   |-- index.js                 # app assembly, boot checks, route mounting
|   |-- blockchain/
|   |   |-- crypto.js            # stableStringify, sha256, ed25519, AES-GCM, CID
|   |   |-- ledger.js           # append-only PoW chain + incremental audit
|   |   |-- engine.js           # the only module that mutates provability state
|   |   |-- rules.js            # pure decision logic (tally, APY, escrow, disputes)
|   |   |-- seed.js             # boot-time genesis, oracle and demo seeding
|   |-- middleware/
|   |   |-- auth.js             # protect, adminOnly, session validation cache
|   |   |-- stepUp.js           # shared step-up re-authentication gate
|   |-- models/                 # 20 files -> 33 core + 25 provability collections
|   |-- routes/                 # 22 groups + web3/ (9 files)
|   |-- utils/                  # 62 modules, the service layer
|   |-- scripts/                # credential notice, hand-off, picture migration
|   |-- Test File/              # 63 *.test.js suites
|   |-- .env.example            # 294-line commented configuration reference
|-- my-react-app/               # ---- FRONTEND ----
|   |-- vite.config.js          # Vite + PWA + optimizeDeps contract
|   |-- capacitor.config.json   # appId com.suppliwise.app
|   |-- android/                # Capacitor project with custom Java
|   |   |-- app/src/main/java/com/suppliwise/app/
|   |   |   |-- MainActivity.java # WebView download handler, MediaStore writes
|   |   |   |-- SafeDownloadName.java # bounded filename derivation from headers
|   |-- public/                 # manifest, icons, robots
|   |-- scripts/                # healDepsCache, verifyDevDeps, icon generators
|   |-- src/
|   |   |-- App.jsx             # routes, guards, error boundary, session sync
|   |   |-- main.jsx            # entry, styles, service worker registration
|   |   |-- api.js              # ~110 API calls, tab-scoped session storage
|   |   |-- api/web3.js         # 62 provability API calls
|   |   |-- auth/authState.js   # single reactive identity source of truth
|   |   |-- hooks/              # useAuth, useSubscription, usePlanDay, useNow
|   |   |-- subscription/       # features.js (entitlement mirror), catalogue.js
|   |   |-- Pages/              # 25 page components
|   |   |-- Components/        # 43 components incl. Web3Panels, admin console
|   |   |-- utils/              # PDF generators, plan day, policy views
```

4.5 Request Lifecycle and Middleware Pipeline

The order of middleware in `server/index.js` is a security control. Requests are admitted by metering before any body byte is buffered.

#	Stage	Component	Function
1	Headers	helmet()	Content-Security-Policy with default-src 'none' and frame-ancestors 'none'
2	Headers	compression()	Level 6 above a 1 KB threshold; honours a client opt-out header
3	Origin	cors()	Explicit origin allowlist with credentials enabled; requests without an Origin header are allowed through for non-browser clients
4	Origin	verifyOrigin	Global CSRF guard: rejects state-changing requests from an untrusted origin
5	Meter	flood guard	Coarse per-IP envelope that meters but never escalates the lockout ladder
6	Meter	rate limiters	Per-group ceilings: auth, session, user, recommendation, AI and admin buckets, each configured in <code>utils/rateLimits.js</code>
7	Meter	lockoutCheck	Escalating lockout ladder — mounted only on <code>/api/auth</code> and <code>/api/admin</code>
8	Meter	bodyBudget	Global concurrent buffering cap; exceeding it returns 503 with <code>Retry-After</code>
9	Parse	rejectOversized	Content-Length pre-check → 413, before <code>express.json()</code> reads the socket
10	Parse	express.json()	1 MB globally; 10 MB on exactly three routes that legitimately accept images
11	Route	router + guards	Group router, then <code>protect</code> / <code>adminOnly</code> / <code>userOnly</code> / <code>requireFeature</code> / <code>requireStepUp</code>
12	Route	handler	Domain logic in the route, business rules in <code>utils/</code>
13	Terminate	error handler	Malformed JSON → 400; 4xx logged through a throttle; 5xx logged in full and reported generically; incomplete responses get <code>Connection: close</code>

WHY THIS MATTERS Only `/api/auth` and `/api/admin` escalate the lockout ladder. Feature traffic — support chat, provability, security dashboard, subscription polling — uses deliberately non-escalating buckets, so ordinary product use can never lock a legitimate member out of their own account.

4.6 Core Data Flows

4.6.1 Assessment to recommendation to plan

1. The member completes the four-step assessment. The browser persists progress in-session so navigation away does not lose it.
2. `POST /api/assessment` stores the assessment, stamps a five-year expiry, and runs severity detection to decide whether a Priority review is warranted.
3. The client requests generation with `POST /api/recommend`.
4. The router dispatches the request to the configured provider and model for the `assessment` purpose.
5. The reply is parsed and passed through `sanitizeStrings()`, which recursively cleans AI output (em dashes, smart quotes, control characters) before it can reach the client.
6. On any failure, the rule-based clinical engine produces a complete plan instead. The member never sees an error state.
7. `PATCH /api/assessment/:id/results` stores the full JSON snapshot, including the wellness baseline, so history replays exactly what was generated.
8. `DashboardMetrics` is created for the new assessment and every previous one is deactivated, giving a clean tracking slate without a migration.
9. The client renders the results and offers “Add to Plan”. Adding writes `IntakeRecord` rows keyed by plan day.
10. Marking a dose taken updates the record, recomputes progress, applies the streak rules and recalculates the wellness score in the same request.

4.6.2 Plan purchase and approval

1. `GET /pricing` is public, so a visitor can compare tiers before creating an account.
2. A signed-in member picks a tier. The server computes and returns the amount; the client never calculates a price.
3. The member submits `POST /api/subscription/requests` with a payment reference and an optional proof-of-payment image.
4. The request is stored with a **server-priced snapshot** of currency, amount and formatted amount, so later price changes cannot alter an existing request.
5. The request enters the administrator queue at `GET /api/admin/subscription-requests`.
6. The administrator approves — optionally overriding the number of days granted — or declines with a note. The decision, actor and timestamp are recorded.
7. Approval writes the paid layer through `subscriptionState.setPaid`, which also publishes on the subscription bus.
8. Every open browser receives the change over SSE at `/api/subscription/stream` and the entitlement store updates without a reload or a sign-out.

4.6.3 Intake, streak and wellness score

Three values are maintained per assessment in `DashboardMetrics`. The **plan day** is defined once (`utils/planDay.js`) as 04:00 to 04:00 in the member's own timezone, labelled by the date it opens on — so the day rolls over at a civilised hour and never at midnight. The **streak** increments once per calendar day on reaching full completion, decreases if the completion is undone before the day closes, and resets at rollover if the previous day was incomplete. Adding a supplement on a day that was already complete breaks completion for that day. The **wellness score** is the sum of an AI baseline (0–30), an adherence component (0–50) and a streak component (0–20), blended so it reacts to today rather than only to history.

4.6.4 Provability anchor

The engine's contract, stated in `blockchain/engine.js`, is: **mutate the database first, then anchor the change**. An anchor failure is logged and never rolls back or fails the business operation — availability is preferred over perfect atomicity. Any value that could identify a person is reduced to a SHA-256 digest over canonical JSON (`stableStringify`) before it reaches the ledger, so the same payload always produces the same hash and no personal data is ever written on-chain.

Chapter 5 · Frontend Subsystem

5.1 Build Toolchain and Configuration

- ``vite.config.js`` uses the function form of ``defineConfig`` so ``cacheDir`` can branch on the command: ``node_modules/vite-build`` for builds and ``node_modules/.vite`` for dev. A production build therefore cannot invalidate the running dev server's optimizer cache.
- The dev server runs on port `**5173**` with ``strictPort``, so it fails loudly rather than drifting to 5174 and confusing API proxying.
- ``/api`` is proxied to ``http://localhost:5000`` during development.
- Security headers (``X-Frame-Options: DENY``, a ``frame-ancestors 'none'`` CSP, ``X-Content-Type-Options: nosniff``, ``Referrer-Policy``) are applied to both the dev server and ``vite preview``.
- ``optimizeDeps.include`` lists every package reached through a dynamic import — ``jspdf``, ``jspdf-autotable``, ``@simplewebauthn/browser``. ****Completeness is enforced by a test****, because a dependency first reached through a dynamic import triggers a re-optimize that invalidates every module URL an open page is already holding.
- ``optimizeDeps.exclude`` keeps Node-only packages out of the browser dependency cache.
- No manual ``rollupOptions`` chunking is configured: splitting comes entirely from route-level ``React.lazy()`` plus Workbox precaching.
- A preflight script (``scripts/healDepsCache.mjs``) repairs the optimizer cache before ``dev`` and ``build``, replacing the documented manual ``rm -rf`` step.

5.2 Routing and Route Guards

Twenty-seven route declarations resolve to twenty-six page components. Every page except the landing page is lazily imported inside a single `Suspense` boundary.

Guard	Behaviour	
LandingRoute	/ renders the marketing page for guests, redirects signed-in members to <code>/dashboard</code> , and sends administrators to <code>/admin</code>	
PublicOnlyRoute	Bounces an admin token to <code>/admin</code> ; permits <code>?add=1</code> so a second account can be added; passes through during an active authentication transition; otherwise sends a token holder to <code>/dashboard</code>	
ProtectedRoute	Refuses an admin token (<code>→ /admin/login</code>); without a user token, redirects to <code>/admin</code> when an admin session exists and the user has <code>*not*</code> just signed out in this tab, otherwise to <code>/login</code>	
AdminProtectedRoute	Requires <code>adminToken</code> in <code>localStorage</code> ; enforces the <code>mustChangePassword</code> redirect, with an explicit exception for the change-password page itself	
Web3PlanGate	For <code>/web3</code> , <code>/marketplace</code> and <code>/governance</code> , renders a <code>PlanLockedCard</code> instead of the page so a sub-tier user never triggers the panels' mount-time fetches	

Path	Page Component	Access
/	HomePage	Public
/login, /signup	LogIn, SignIn	Public (redirect when signed in)
/forgot-password	ForgotPassword	Public, unauthenticated
/reset-password?token=	ResetPassword	Public — the URL token, not the session, authorises; deliberately not wrapped in <code>PublicOnlyRoute</code>
/pricing, /plans	PricingPage	Public by design
/verify, /verify/:code	VerifyPage	Public — recomputes anchored proofs
/share/:token	SharePage	Public — read-only, revocable profile link
/admin/login	AdminLogin	Public, self-guards on an existing admin token
/dashboard	DashboardPage	Member
/assessment	AssessmentPage	Member
/results	ResultsPage	Member
/recommendations	RecommendationsPage	Member
/track-intake	TrackIntakePage	Member
/insights	InsightsPage	Member + Insights entitlement
/history	HistoryPage	Member
/profile	ProfilePage	Member
/support	SupportChatPage → SupportInbox	Member
/web3	Web3HubPage → 8 panels	Member + Provability entitlement
/marketplace	MarketplacePage	Member + Marketplace entitlement
/governance	GovernancePage	Member + Governance entitlement
/admin	AdminDashboard (8 tabs)	Administrator
/admin/assessment-management	AssessmentManagement	Administrator
/admin/change-password	AdminChangePassword	Administrator (forced)

5.3 Page Inventory

Twenty-five page components, 35,002 lines of JSX in total. The three largest — AdminDashboard (5,256), AssessmentPage (2,658) and PricingPage (1,931) — are referenced in the oral defense as the project's largest single implementation surfaces.

Page	Lines	Responsibility
AdminDashboard	5,256	The entire administrator console shell: eight tabs, idle countdown, token refresh policy, trend charts, overview analytics
AssessmentPage	2,658	Four-step wizard — demographics, health, symptoms, current supplements — with unit conversion, clamping, spam detection and prefill
PricingPage	1,931	Public catalogue, checkout sheet, proof-of-payment submission, downgrade and cancellation modes, currency preference
ProfilePage	1,638	Profile and billing hub: avatar and banner upload, name/DOB edit, security controls, account switcher, plan requests
HistoryPage	1,346	Paginated assessment history with PDF export, delete and plan-aware page depth
TrackIntakePage	1,143	Today's supplements tracker: slot grouping, dose windows, calendar, weekly adherence, bulk tick, day rollover
DashboardPage	1,397	Signed-in home: today's plan, action cards, mark-taken intake, energy level, wellness score
ResultsPage	852	The generation moment: full plan, schedule, lifestyle advice, confidence bars, evidence modals, PDF export
RecommendationsPage	819	Plan manager: priority tabs, filters, add/remove supplement, detail overlays
LogIn	819	Sign-in with typo-TLD email detection and passkey sign-in
InsightsPage	693	Insights and analytics, plan-gated
ChatAssistant	666	Floating assistant pill and panel with a dependency-free markdown renderer
MarketplacePage	739	Listings, escrow orders, delivery confirmation, disputes and juror vote
GovernancePage	546	Stake-weighted proposals and the knowledge base
ForgotPassword	629	Unauthenticated recovery hub: emailed link, authenticator, passkey, backup code, recovery email
SignIn	520	Registration with CAPTCHA and shared name validation
AdminLogin	363	Admin alias + password → TOTP challenge
AssessmentManagement	574	Admin assessment browser with read-only views, modify modal and PDF export
ResetPassword	454	Token redemption with explicit load/dead/throttled/form/done phases
Web3HubPage	66	Tab shell hosting the eight provability panels
VerifyPage / SharePage	200 / 132	Public proof verification and public shared-profile views
HomePage	232	Landing page with plan cards sourced from the catalogue
SupportChatPage	35	Thin /support shell around SupportInbox
AdminChangePassword	284	Server-enforced forced-password-change screen

5.4 Component Inventory

Forty-three components across six functional groups.

Group	Components
Navigation and account	Navbar (sticky surface, notifications bell, account menu), AccountSwitcher (multi-account panel), ProfileActionsMenu, ConfirmLogoutModal, SessionExpiryModal (admin idle countdown), ProfileAvatarImage
Member shell	Toast, ConfirmModal, UpgradeModal (reusable paywall), PlanLockedCard (value-first lock state), Web3PlanGate, UserNotifications, PWAInstallPrompt
Provability panels	Web3Panels/w3ui (shared kit), WalletPanel, RewardsPanel, LedgerPanel, PrivacyPanel, AiProofPanel, EcosystemPanel, SupplyPanel, ExplorerPanel
Account security	ProfileSecurityControls (container), PasskeyPanel, RecoveryPanel, StepUpDialog, SessionActivity
Admin console	AdminTopbar, AdminSubscriptionRequests, AdminSubscriptionCancels, AdminSubscriptionPanel, AdminSupportChats, SecurityStatus (the live monitor), AssessmentResultsDisplay, ReadOnlyAssessment, ModifyAssessmentModal, ImageLightbox
Shared domain	SupplementDetail / EvidenceInfoModal, SupportInbox, MyPlanRequests / MyPlanCancels, AdminProtectedRoute, SessionRevalidator

5.5 Client State and Session Model

The frontend has no global state library. Authority is split deliberately:

- `src/auth/authState.js` is the single reactive source of truth for identity. It owns the event name, the per-tab storage keys, the shared account directory and the administrator-token key, so no component invents its own notion of "signed in".
- Sessions are ****per-tab `sessionStorage`**** with a shared account directory in `localStorage` and a `BroadcastChannel` for cross-tab handoff. Several accounts can be signed in simultaneously, each in its own tab.
- **useAuth deliberately ignores cross-tab `storage` events:** another tab signing out must not sign this tab out.

- ``useSubscription`` (524 lines) is the entitlement store: SSE from ``/api/subscription/stream``, a ``BroadcastChannel`` relay for other tabs, a local expiry timer, throttled and focus-driven refreshes, and ``canAccess`` / ``canAccessTier``. It is **keyed by token**, so an in-place account switch can never hand one account's plan to the next.
- Reading and formatting logic is extracted into pure modules with their own tests — ``recommendationView``, ``adminOverview``, ``slotSchedule``, ``timeSlots``, ``planDay``, ``passwordPolicy``, ``nameValidation``, ``greeting``, ``dates``. This is why the client test suite can cover business presentation without a DOM framework.

DESIGN NOTE A meaningful share of the client tests are **source-level**: they read a `.jsx`, `.css` or config file and assert a pattern. This is deliberate — `node --test` cannot import JSX or CSS, so wiring is verified by reading the source. Tests such as `planDayWiring`, `greetingWiring`, `navbarSticky`, `navbarMotion`, `navbarSurface` and `optimizeDepsInclude` all work this way. Explain this in the defense; it is a deliberate strategy, not an oversight.

5.6 PWA and Android Packaging

- The PWA is configured with ``registerType: 'autoUpdate'``, a standalone display, portrait-primary orientation, themed icons at 192 and 512 px in ``any`` and ``maskable`` forms, and two app shortcuts (Dashboard, New Assessment).
- **The service worker is disabled in development.** A worker in front of Vite serves stale bundles; ``main.jsx`` actively unregisters workers and clears caches in dev.
- Runtime caching is limited to Google font hosts. ``/api`` is **never** cached — health data must not be served from a cache.
- Capacitor 8 wraps the built SPA with app id ``com.suppliwise.app``.
- Custom native code (``MainActivity.java``, 210 lines) installs a ``WebView`` ``DownloadListener`` so a generated PDF is actually written to device storage: ``data:`` URLs are base64-decoded and written through ``MediaStore.Downloads`` on Android 10+, with a canonical-path-checked ``File`` fallback plus ``MediaScannerConnection`` and ``DownloadManager`` on Android 9 and below. ``http(s)`` URLs route through ``DownloadManager`` into ``DIRECTORY_DOWNLOADS``. Any other scheme is rejected.
- ``SafeDownloadName.java`` (97 lines) is the security boundary for that sink: it parses ``Content-Disposition`` (RFC 5987 ``filename*`` first, then ``filename``), URL-decodes, strips path separators and control characters, rejects ``.`` and ``..``, and clamps the name to 120 characters while preserving a short extension. It has its own JVM unit test.
- The output APK is forced to ``SuppliWise.apk`` at ``android/app/build/outputs/apk/debug/SuppliWise.apk``.

Chapter 6 · Backend Subsystem

6.1 Bootstrap and Boot-Time Checks

`server/index.js` (718 lines) refuses to start rather than start insecurely. The following conditions each throw during module load:

Check	Condition
JWT secret present	<code>JWT_SECRET</code> is defined and is not the documented placeholder value
JWT secret strength	<code>JWT_SECRET</code> is at least 32 characters
Debug switches off	In production, <code>ALLOW_DEV_OTP_RESPONSE</code> is not <code>true</code>
Public URL	In production, <code>PUBLIC_WEB_URL</code> is non-empty (reset-link origin)
Trusted origins	In production, <code>assertProductionConfigured()</code> confirms a trusted-origin allowlist is configured
Secret box	In production, an encryption key is available and its source is <code>TOTP_ENCRYPTION_KEY</code> — a JWT-derived fallback is rejected
WebAuthn	In production, a real relying-party id and at least one <code>https</code> origin are configured

Five background jobs start after MongoDB connects, each non-blocking and each failing soft so a degraded subsystem does not prevent boot:

- Provability genesis seeding (`'blockchain/seed.js'`) — the endpoints lazy-initialise if seeding has not completed.
- A whole-chain integrity pre-computation (`'blockchain/ledger.audit()'`), which logs `'chain verified at boot'` or `'CHAIN INTEGRITY FAILED'`.
- A live SMTP configuration check (`'utils/email.verifyEmailConfig()'`).
- A loud warning if self-serve purchase grants paid plans without review.
- A ****secret-free**** summary of the auth-security configuration: passkeys on/off, TOTP key source and key ids, and the number of trusted origins.

NOTE `app.set('trust proxy', 1)` is enabled **only** when `TRUST_PROXY=true`. It is off by default so a spoofed `X-Forwarded-For` header cannot poison `req.ip` — which drives rate limiting, lockout accounting and login evidence.

6.2 Route Groups and API Surface

220 endpoints across 22 route groups mounted under `/api`, plus 9 Web3 sub-routers mounted at `/api/web3`.

Group	Path	Guard
Authentication	<code>/api/auth</code>	Rate-limited; <code>lockoutCheck</code> on credential paths; <code>sensitiveLimiter</code> on 12 credential routes
Password reset	<code>/api/auth/password-reset</code>	Enumeration-safe; four separate limiters (request, redeem, factor, passkey)
Passkeys	<code>/api/auth/passkeys</code>	<code>verifyOrigin</code> ; <code>requireStepUp</code> on register and delete
TOTP	<code>/api/auth/totp</code>	<code>verifyOrigin</code> ; <code>requireStepUp</code> on every mutation
Recovery codes	<code>/api/auth/recovery-codes</code>	<code>requireStepUp</code> to generate, regenerate
Sessions	<code>/api/auth/sessions</code>	<code>verifyOrigin</code>
Assessment	<code>/api/assessment</code>	<code>protect</code>
Subscription	<code>/api/subscription</code>	<code>protect</code> for purchase and requests; public for <code>/plans</code> ; SSE stream authenticated by a token shim
AI	<code>/api/recommend</code> , <code>/api/chat</code> , <code>/api/polish</code> , <code>/api/supplement-detail</code>	<code>protect</code> ; <code>requireFeature('chat')</code> for chat
Support	<code>/api/support-chat</code> , <code>/api/admin/chats</code>	<code>protect</code> ; <code>adminOnly</code> for the admin inbox
Dashboard / Insights	<code>/api/dashboard</code> , <code>/api/insights</code>	<code>protect</code> ; <code>requireFeature('insights')</code> on insights
Notifications	<code>/api/notifications</code>	<code>protect</code>
Account security	<code>/api/security</code>	<code>protect</code> , administrators excluded
Provability	<code>/api/web3</code>	<code>userOnly</code> + path-keyed plan gate; only <code>/verify/:code</code> , <code>/verify/:code/qrcode</code> and <code>/share/:token</code> are public
Admin	<code>/api/admin</code>	<code>protect</code> + <code>adminOnly</code> on every route

DESIGN NOTE The pre-session recovery-code sign-in route lives in `routes/securityRedeem.js`, which is deliberately **not** mounted in `index.js`. It is registered by `routes/security.js` ***before*** that router applies `protect`, which is the only way a pre-session endpoint can exist inside the security route family. This ordering is easy to “clean up” by accident — flag it in the defense as intentional.

NOTE The Web3 plan gate is decided **once**, in `routes/web3/index.js`, keyed on the request path. All seven sub-routers mount at `/`, so a `router.use()` guard inside any one of them would gate every path. Each sub-router keeps its own guard as defence in depth.

6.3 Data Model

MongoDB with Mongoose 8. No migration tooling is required because the AI result snapshot is a schemaless Mixed field — the model output shape can change without a schema migration.

Collection	Model File	Purpose and notable fields
users	User.js	Identity, argon2id password, profile and banner pictures, 2FA method with encrypted seed, recovery email, login metadata, currentSessionId, subscription record
assessments	Assessment.js	All assessment fields plus the full aiResults JSON snapshot including the wellness baseline; priority flag and reasons; five-year expiresAt
intakerecords	IntakeRecord.js	Per-day dose records with dayKey, taken state and timestamps
dashboardmetrics	DashboardMetrics.js	Wellness score, streaks, adherence and energy level per assessment
sessions	Session.js	One-per-account sessions with device attribution, optional absolute expiry, revocation stamp
passkeys	Passkey.js	WebAuthn credentials with a globally unique credentialId, signature counter and transports
authchallenges	AuthChallenge.js	WebAuthn ceremonies with attempt budget and a TTL index
mfatransactions	MfaTransaction.js	The short-lived second-factor budget issued at the password step; TTL indexed
backupcodes	BackupCode.js	SHA-256 hashes of single-use recovery codes, batched so a whole set can be invalidated atomically
passwordresettokens	PasswordResetToken.js	SHA-256 reset grants with a Mongo TTL index and a single-use conditional claim
securityevents	SecurityEvent.js	Append-only per-user security history over a closed enum of event types
usernotifications	UserNotification.js	The member notification inbox (mutable, with soft dismissal)
adminevents	AdminEvent.js	The administrator notification stream over a typed enum
subscriptionrequests	SubscriptionRequest.js	Purchase requests with a server-priced snapshot and an optional proof-of-payment image
subscriptioncancelrequests	SubscriptionCancelRequest.js	Cancellation requests in immediate or reviewed mode
chatthreads	ChatThread.js	Support conversations with category, status, denormalised last-message fields and unread counters
chatmessages	ChatMessage.js	Append-only messages capped at 2,000 characters; the author is derived from the session, never the request body
adminaccounts	AdminAccount.js	Administrator identity projection with non-selected password hash and TOTP secret
supplementdetails	SupplementDetail.js	Supplement guides keyed by normalised name, with per-profile variants
25 provability collections	Web3.js	Blocks, chain audit checkpoint, wallets, config parameters, supply batches, listings, orders, disputes, proposals, knowledge posts, reward events, NFTs, loyalty codes, data shares, storage objects, health anchors, share links, recommendation anchors, trials, trial consents, oracle feeds, experts and bookings

Key behaviours enforced by hooks and indexes:

- Pre-save hooks stamp `passwordChangedAt` in the same write as the new hash, so the record can never disagree with itself.
- Deleting a user cascades to assessments, intake records, dashboard metrics, subscription requests, cancellation requests, chat threads and chat messages. Thread ids are collected first so an interrupted cascade cannot orphan a transcript.
- `PasswordResetToken` and `AuthChallenge` and `MfaTransaction` carry Mongo TTL indexes, so expiry is a database guarantee rather than a cron job.
- `IntakeRecord` is indexed on `{user, assessment, dayKey}` and `{user, dayKey}`; `Assessment` on `{user, createdAt: -1}`; `DashboardMetrics` on `{user, assessment}` and `{user, isActive}`.
- Reward idempotency is a **unique index** on `(user, kind, refId)`, so a double click cannot mint twice.

6.4 Utility and Service Layer

Sixty-two modules in `server/utis/` hold the business rules. Keeping them out of route handlers is what makes the rules independently testable and reusable across endpoints.

Domain	Key Modules
Passwords	passwordRules (the single policy, served over the wire so the UI cannot drift), password (argon2id with transparent bcrypt upgrade), passwordReset, passwordRecovery
Sessions	sessions (server-authoritative store), userSession (the no-time-based-expiry policy in one place), adminSession (the single idleExceeded decision shared by middleware and routes), authFlow (one shared sign-in completion used by every second-factor path)
Second factor	totp, totpSecret (the only module that reads or writes a TOTP seed), webauthn, mfaTransaction, secretBox, strongestFactor
Step-up	middleware/stepUp.js — purpose-pinned, session-bound, user-bound, algorithm-pinned 5-minute token carried in X-Step-Up
Abuse control	lockout (escalating ladder), floodGuard (pre-parse metering and body budget), rateLimits (all ceilings as validated configuration), captcha

Domain	Key Modules
Entitlements	entitlements (the one source of truth, served to the frontend), planCatalogue, subscriptionState (paid + override layers), subscriptionBus, subscriptionRequests, subscriptionCancels, paymentInstructions, currency
Domain logic	severity (rule-based severe-case detection), priorityFlagging (AI second opinion), priorityGate, planDay, intakeWindows, dailyScheduleSlots, wellnessScore, assessments
AI	aiRouter, aiProviders, chatSafety, systemDetection, supplementGuideStore, recommendationPlainLanguage
Infrastructure	origins (one trusted-origin list for both CORS and the CSRF guard), envFile (re-readable .env, reports variable names only), geo, device, cache, email, emailValidation, sanitize, nameValidation, transientError, bigDocuments
Admin	adminAccounts (reads and writes the packed ADMIN_ACCOUNTS value), adminCredentialNotice, adminCredentialHandoff, overviewAnalytics, securityAudit, attack_probes

CAUTION Known drift to disclose. server/package.json declares npm run send-admin-credentials pointing at scripts/send-admin-credentials.js, which is not present in the repository, so that script name fails. The working equivalents are the POST /api/admin/admins/credential-handoff route and utils/adminCredentialHandoff.js, both of which exist and are tested. Disclosing this yourself is far stronger than being asked about it.

Chapter 7 · Artificial Intelligence Subsystem

7.1 Purpose and Design Constraints

The AI layer produces every piece of personalised health content in the product: the assessment plan, the supplement deep-dive, the polished patient text, the chat assistant, the severity second opinion and the administrator's threat prediction. It was built under three constraints that shaped every decision below.

- 1. **Availability.** The product must return a plan. A third-party outage, an exhausted balance or a rate limit cannot become a user-facing error.
- 2. **Honesty.** The output is health-adjacent. It must not invent symptoms, invent severity, or claim a finding the user did not report.
- 3. **Privacy.** Only the minimum necessary leaves the process, and a health record is never sent to a provider.

7.2 Provider Routing Table

Features are dispatched through a single table (`utils/aiRouter.js`) rather than by hard-coding a vendor inside each route. To change the split, edit `AI_ROUTES`; nothing else needs to know.

Purpose	Provider	Default Model	Used For
assessment	OpenRouter	deepseek/deepseek-v4-flash-0731	The full wellness plan and supplement recommendations
polish	OpenRouter	deepseek/deepseek-v4-flash-0731	Rewriting free-text patient notes into clinical language
supplementDetail	OpenRouter	deepseek/deepseek-v4-flash-0731	The personalised “tap for details” deep dive
chat	Anthropic (→ OpenRouter fallback)	claude-haiku-4-5-20251001	The AI chat assistant
priorityFlagging	Anthropic	claude-haiku-4-5-20251001	Second opinion on assessment severity
systemDetection	Groq	openai/gpt-oss-120b	Threat prediction for the administrator Security Center

The split follows workload character. Features that generate a member's health content stay on one provider for consistency of voice and citation style. Short, structured, latency-sensitive calls that run on a timer go to the fastest provider. The chat assistant and severity flagging go to a third.

WHY THE FALLBACK EXISTS `chat` is the one purpose with a fallback, and this is load-bearing rather than decorative. A provider key can be perfectly valid and still unable to serve a single completion — an exhausted Anthropic balance authenticates fine and returns a 400 that the model-list health probe cannot see. Without a fallback, moving chat to an unfunded account would put every user on the canned offline reply. The primary is always tried first, and the response's `source` field names the provider that actually replied.

THE DRIFT THIS TABLE PREVENTED Each call site used to hard-code its own URL and model string, and they had already drifted apart: `chat.js` and `polish.js` requested `deepseek-v4-flash` while the admin panel reported `deepseek-v4-flash-0731` for the same key. OpenRouter's probe cannot catch this — its model list is roughly 752 KB and will not fit inside the 5-second probe budget — so it would surface only as a failed user request. A single table removes the possibility rather than the instance.

Two implementation details are worth stating in the defense:

- Anthropic's Messages API is **not** OpenAI-shaped: ``system`` is a top-level field, there is no ``role: 'system'`` message, and adjacent same-role turns are rejected. The router builds a different request body and reads a different response shape per provider (`wire: 'anthropic'`), lifting ``system`` out of a conversation, merging same-role turns and dropping a leading assistant turn — none of which a natural chat history satisfies.
- Provider-specific tuning with no cross-provider equivalent (OpenRouter's `reasoning.effort`) is passed as `extraBody` and applied only to the OpenAI-shaped wire, because Anthropic answers an unknown top-level key with a 400.

OPERATIONAL WARNING Model ids depend on the key that is configured. Groq and Anthropic each serve different model lists per tier, so a model the key cannot reach fails the first real call — not the health check. Conversely, **a valid key is not enough**: an account with no credit authenticates fine and serves the model list, so the health card reports “Reachable” while every completion fails and the feature silently never runs. That is why the admin AI panel reports the *last real call* on a separate line. That row is the one to read.

7.3 Recommendation Pipeline

`POST /api/recommend` runs three stages in order:

- 1. **Model generation.** The router dispatches to the configured provider and model for the ``assessment`` purpose, requesting a structured JSON wellness plan of up to 20 personalised supplements.
- 2. **Sanitisation.** `sanitizeStrings()` recursively cleans the model output — em dashes, smart quotes, unusual control and special characters — before it can reach the client.
- 3. **Deterministic fallback.** If the model path fails for any reason, a rule-based clinical engine produces a complete plan covering all age groups, medications, conditions, symptoms, diet type, lifestyle and pregnancy status.

The prompt carries explicit quality rules that the client also enforces in presentation:

- Consider **only** information the user provided — never invent a symptom or a severity.
- Apply severity labels only to symptoms actually reported.
- Build `triggeredBy` from reported data alone, falling back to “General wellness”.
- `ReturnSystemContext` as `null` when there is nothing to reference.
- Respect diet type, allergies and medical conditions strictly in meal generation.

Presentation layers then enforce the same honesty contract. The client sorts high → medium → low, then by confidence score, and moves supplements already in the plan to the bottom. Each card shows the **exact** reported condition, symptom or goal that triggered the recommendation, plus a condition callout (“Since you reported Diabetes with moderate fatigue...”), an animated confidence bar, dosage, timing, interaction warnings and a professional disclaimer. Expanding the card reveals the evidence with RRL citations and PMIDs, food sources, side effects and safe limits.

NOTE All recommendation language is **possibility language** — “may support”, “evidence suggests”. This is not medical advice, and the generated PDF carries a consult-a-professional notice. Keep this consistent in every written chapter and slide.

7.4 Priority Assessment Flagging

When a member submits an assessment, the system decides whether it warrants a Priority review. This is a clinical-safety decision, and its design is the most defensible piece of the AI subsystem.

utils/severity.js is a deliberate **rule engine**: fixed lists of red-flag symptoms, critical conditions and emergency phrases. It is auditable, instant, and cannot be talked around by the wording of a free-text field. Its own header records why it was tightened — it once flagged nearly everything, because the word “severe” in a generic AI disclaimer matched.

A fixed list only finds what someone thought to write down, so utils/priorityFlagging.js adds a second opinion from Anthropic that reads the same submission as free text, looking for clinical concern the keyword list missed. The governing invariant is:

```
finalFlag = ruleVerdict.flagged || aiEscalated

// The AI layer can ONLY add a flag. It can never remove one.
```

The asymmetry is deliberate and is the single best answer in the AI chapter:

- A missed severe case is a clinical risk that nobody reviews.
- A spurious flag pauses a member’s new assessments — a real cost this codebase has already been bitten by, which is why `priorityGate.js` carries a documented “panicked user” scenario with no exit.

An AI-only flag therefore requires **all** of: the rules did not flag, confidence ≥ 80, and at least one citable reason. Any failure — no key, no credit, rejected, timeout, unparseable reply, a reply that fails validation — leaves the rule verdict completely untouched. analyzeSeverity is deliberately left synchronous, pure and unmodified as that floor.

PRIVACY Only health fields are sent — never a name, email or picture. Patient free text is stripped of emails, phone numbers and handles, and instruction-override phrases are defanged, because feelingDescription is written by whoever filled in the form.

7.5 Chat Assistant and Safety Guardrails

- A floating “Ask AI” bubble is available on all member pages and deliberately hidden on authentication pages, `/support` and the entire administrator area.
- Server-side safety (utils/chatSafety.js) bounds input length and turn count, builds the system prompt, and keeps answers inside the health, nutrition and SuppliWise domain — off-topic requests are politely declined rather than answered.
- The system prompt includes typo tolerance for health and supplement terms, so “magnezium” is handled gracefully.
- A local fallback reply is available when no provider answers, and `ChatInputError` maps malformed requests to a 400 rather than a 500.
- The assistant is an **Ultimate entitlement** enforced server-side via `requireFeature('chat')`, not merely hidden in the UI.
- The panel auto-scrolls to top on open and offers a scroll-to-bottom button while the user reads history. It contains a small dependency-free markdown renderer rather than pulling in a full parser.

7.6 Deterministic Clinical Fallback

The rule-based engine is the reason the product has a hard availability guarantee. It derives recommendations from the assessment directly — allergies, existing supplements, blood-test interpretation, age band, medications, conditions, symptoms, diet, lifestyle and pregnancy — and it emits plain-language reasons and evidence (recommendationPlainLanguage.js) rather than placeholder text. Because it requires no network, it is also what makes the system demonstrable in a venue with unreliable connectivity — a practical point to raise before a panel demo.

7.7 AI Privacy Controls

Control	Implementation
Data minimisation	Only health fields are transmitted. Names, emails, profile pictures and contact details never reach a provider.
Free-text scrubbing	Emails, phone numbers and handles are stripped; instruction-override phrases are defanged before submission.
Threat-prediction redaction	Probe details are redacted — emails, IPs, JWTs, API keys and long hashes replaced — before they leave the process. The model receives probe status and a length-capped detail, never user data and never a credential.
Output validation	Severity is clamped to nominal watch elevated severe; categories to a fixed vocabulary; every string is length-bounded and stripped; a malformed reply is discarded whole rather than half-applied.
Ledger privacy	Anything sensitive is reduced to a SHA-256 digest over canonical JSON before anchoring. No personal data is written on-chain.
Cache discipline	Threat verdicts are cached for five minutes and recomputed only when a probe status actually changes; supplement guides are cached per reader with single-flight de-duplication.

Chapter 8 · Subscription and Entitlements

8.1 Subscription Tiers

The stored plan identifiers (`free`, `monthly`, `annual`, `custom`) are the contract for the entire entitlement system and are never renamed — only the display names change. A fifth **Team** band shares the Premium tier across 2–500 seats with a single computed total.

Tier	Stored id	Unlocks
Free	free	Health assessment, supplement recommendations, daily intake
Deluxe	monthly	+ Insights and analytics, PDF report export, and the entire provability layer
Premium	annual	+ Priority assessment review, five-year record history
Ultimate	custom	+ AI chat assistant — every feature the system has
Team	annual + seats	Premium shared across 2–500 seats, one computed total

History page depth also scales with plan: 5 assessments per page on Free, 10 on Deluxe, and 20 on Premium and above.

8.2 Billing Model

- A purchase is exactly **30 days** (`STANDARD_PERIOD_DAYS`). Nothing renews automatically.
- Buying again **extends** the existing window rather than restarting it, so no paid day is ever discarded.
- A plan change applies immediately, with no sign-out.
- When the days run out the account returns to Free. **Assessments and history are kept, not deleted.**
- There is no refund mechanism. Every “refund” in the codebase is provability-marketplace escrow, which is a different product.
- Base prices live in PHP on the server and every other currency is derived there, so a displayed price can never disagree with the charged price.

8.3 Purchase and Approval Workflow

The pricing page is **public** — a visitor must be able to compare plans before creating an account. Purchase, however, requires a session and a human decision.

Step	Actor	Action
1	Visitor	Opens <code>/pricing</code> and compares plans in the detected currency
2	Member	Selects a plan; the server computes and returns the amount
3	Member	Submits a plan request with a payment reference and an optional proof-of-payment image; the server snapshots currency, amount and formatted amount onto the request
4	Administrator	Reviews the queue at <code>GET /api/admin/subscription-requests</code> , optionally overrides how many days are granted, then approves or declines with a note
5	System	Approval writes the paid layer and publishes on the subscription bus; the decision, actor and timestamp are recorded
6	Member	The granted plan is live immediately, pushed over SSE without a reload

Cancellation requests run through the same review model in the opposite direction, with an `immediate` mode and a `review` mode, and each request stores the mode so the client can label the commit button correctly.

8.4 Entitlement Enforcement

- The **server** is the single source of truth (`utils/entitlements.js`). The frontend mirror (`src/subscription/features.js`) exists only to render gates and upgrade prompts instantly.
- Feature lookup **fails closed**: unknown keys and prototype keys (`constructor`, `__proto__`, ...) return `null`, so every gate answers “locked”.
- The subscription state is **two-layered** — the member's own paid subscription plus an administrator override — with a frozen `restore` snapshot and an append-only history record. The administrator UI shows effective state, paid state and override separately, and “restore original” returns the member's exact prior paid state.
- Plan changes reach open browsers over **SSE** at `/api/subscription/stream`, with a focus refresh, an expiry watch and a cross-tab relay.
- The entitlement store is keyed by token, so an in-place account switch cannot hand one account's plan to the next.
- A `PlanLockedCard` replaces dead-end lock screens. It states the value first, shows the literal plan gap, and lists what the upgrade actually buys.

DESIGN NOTE The SSE stream cannot set request headers, so `/api/subscription/stream` authenticates through a dedicated token shim rather than the standard bearer header. This is a real constraint of `EventSource` and worth naming if a panel asks how live updates are delivered.

8.5 Currency Resolution

Detection order is fixed and server-side: saved preference → `?currency=` query parameter → `Accept-Language` header → IP geolocation → PHP base. The picker always states where the displayed number came from (“based on your location”, “your chosen currency”, and so on), so the member can tell a converted figure from an original.

Chapter 9 · Provability Layer (Web3)

9.1 Positioning and Scope

SuppliWise runs a **self-contained, simulated proof-of-work chain inside the application**. There is no external chain, no gas, no funded wallets and no RPC endpoint. State this plainly and early in any written chapter — overclaiming here is the fastest way to lose a technical panel's trust in the rest of the work.

STATE THIS EXACTLY MongoDB is the source of truth. The chain is a tamper-evident audit anchor, not a second source of truth. The contract in `blockchain/engine.js` is: mutate the database first, then anchor the change — an anchor failure is logged and never fails or rolls back the business operation. There is no consensus across nodes, no difficulty adjustment, no block reward, no miner competition and no fork-selection rule, because there is only ever one process. What is real is the cryptography: ed25519 signatures, AES-256-GCM encryption, SHA-256 content addressing, hash-chained blocks and deterministic re-verification.

Privacy is non-negotiable within this layer. Personal data is never written on-chain. Anything sensitive is reduced to a sha256 digest over canonical JSON (`stableStringify`), so the same payload always produces the same hash.

9.2 Chain Architecture

Module	Responsibility	Notes
<code>blockchain/crypto.js</code>	Primitives: <code>stableStringify</code> , <code>hashPayload</code> , <code>sha256Hex</code> , <code>ed25519 generateIdentity / sign / verify</code> , <code>AES-256-GCM encrypt / decrypt</code> , <code>IPFS-style contentId</code> , <code>dayKey</code> , <code>round2</code>	Pure functions; the basis of every digest in the system
<code>blockchain/ledger.js</code>	Append-only proof-of-work ledger: block minting, hash chaining, a serialised append queue, and <code>audit()</code>	Difficulty 3 (<code>CHAIN_DIFFICULTY</code>) $\approx$ 4,096 hashes, sub-millisecond. Block shape is <code>{index, timestamp, prevHash, nonce, hash, txs}</code>
<code>blockchain/engine.js</code>	The business layer over the ledger and the models: wallets, rewards, staking accrual, escrow settlement, NFT minting, DAO execution	The only module permitted to mutate provability state. Enforces the database-first, anchor-second contract
<code>blockchain/rules.js</code>	Pure, I/O-free decision logic: <code>tallyProposal</code> with quorum and strict majority, <code>streak math</code> , reward amounts, <code>escrowSplit</code> , juror quorum and majority, proof-of-work difficulty	No database access, therefore fully unit-testable
<code>blockchain/seed.js</code>	Genesis bootstrap: configuration parameters, oracle feeds, sample experts, trials, listings and supply batches	Also serves as the lazy initialiser if boot seeding has not completed

Economic parameters live in a single document (`DEFAULT_PARAMS` in `models/Web3.js`) — every reward rate, fee and quorum threshold in one place, and DAO-governable at runtime. There is no magic number for an economic rule anywhere else in the codebase.

9.3 Feature Inventory

Twenty features, each mapped to its implementing file and the check that proves it in `BLOCKCHAIN_FEATURES.md`.

#	Feature	What it does
1	Supply chain tracking	Forward-only batch journey (<code>raw-sourcing</code> → ... → <code>delivered</code>) with a public QR scan view that recomputes every proof
2	Verifiable certifications	Lab, organic, non-GMO, third-party and GMP digests anchored with their block index
3	Smart-contract escrow	Buyer funds locked in <code>sw_system_escrow</code> ; delivery confirmation executes the split (seller paid, minus DAO fee)
4	P2P marketplace	Verified listings and orders priced against oracle feeds
5	Decentralized identity (DID)	ed25519 wallet, <code>did:suppliwise:<userId></code> , PKCS#8 PEM private-key export restricted to the owner
6	User-owned health ledger	Rolling digest over real assessments plus intake volume, with a signed portable credential export
7	Data sovereignty	Coarse anonymised dataset (age band, gender, volume bands) shared with on-chain consent — and revocation that destroys the payload
8	Decentralized storage	AES-256-GCM at rest, <code>bafy...</code> content address, recomputed on fetch as tamper evidence
9	WELL token rewards	Streak-scaled daily check-in, intake, assessment and data-share rewards, idempotent per day
10	Achievement NFTs	Eligibility computed server-side from real activity, minted soulbound, concurrency-safe
11	Staking	APY-pro-rated yield, threshold-gated premium unlock, partial unstake
12	Loyalty programme	Burn WELL for a one-time <code>LOY-...</code> code that discounts a marketplace order
13	DAO governance	Stake-weighted, quorum-gated proposals that actually apply the parameter on passage
14	Community knowledge base	Publishing and upvote rewards with caps; self-upvotes and double votes rejected
15	Dispute resolution	Jurors drawn from staked non-party wallets; parties can never vote on their own dispute
16	Interoperable health profile	Time-boxed, revocable share link readable by a clinician with no account
17	Verifiable AI recommendations	Inputs, outputs and logic version hashed separately; any drift fails re-verification
18	Clinical trial consent	Exact terms hashed and anchored; opt-in and withdrawal both recorded
19	Oracle feeds	Six feeds re-derived deterministically per day and anchored only on change

#	Feature	What it does
20	Expert consultations	Tokenised booking at rate × hours into <code>sw_system_expert_pool</code> , refundable on cancellation

Feature 17 deserves emphasis in the defense. Because the inputs, the outputs and the logic version are hashed **separately**, a provider that silently changes its model is detectable: re-verification fails, and the user is told the plan can no longer be proven. That is a stronger claim than “we stored the result”, and it is the one feature that directly answers the AI trust problem in Chapter 1.

9.4 Foundation Guarantees

Concern	Guarantee
Ledger	Append-only proof-of-work chain of {index, timestamp, prevHash, nonce, hash, txs} with difficulty 3
Integrity	GET /api/web3/chain/verify recomputes every header hash and prevHash linkage; verification is incremental from a persisted checkpoint and also runs in full at boot
Key custody	Private keys sealed in an AES-256-GCM envelope keyed from <code>JWT_SECRET</code> ; key export stamps <code>keyExportedAt</code>
Reward safety	A unique index on (user, kind, refId) means a double click can never mint twice
Economics	All rates, fees and quorum thresholds in one DAO-governed document
Public surface	Only GET /verify/:code, /verify/:code/qr and /share/:token are public. Everything else requires a member session and is refused to administrator identities
Client gating	/web3, /marketplace and /governance render a plan gate instead of the page, so a sub-tier user never triggers the panels' fetches

LIMITATION TO DISCLOSE The verification endpoint checks header hashes and `prevHash` linkage. The human-readable payload of a transaction is authenticated by its own `txHash`, not folded into the block header. Additionally, the chain audit checkpoint that makes verification cheap is a document in the same database, so it is an internal consistency check rather than an external trust anchor. Naming both of these yourself demonstrates exactly the self-awareness a capstone panel rewards.

9.5 Public Attack Surface

Three endpoints are reachable without a session, and each is deliberately designed to be safe when read by a stranger:

- `GET /api/web3/verify/:code` — recomputes every anchored proof for a supply-chain batch. Read-only, and it verifies rather than trusts.
- `GET /api/web3/verify/:code/qr` — the same data rendered as a QR code for a physical label.
- `GET /api/web3/share/:token` — a read-only, time-boxed, revocable health profile. The token is opaque, the payload is coarse, and the creator can revoke access at any time.

Chapter 10 · Security Architecture

10.1 Defence Model

Security is implemented as ordered layers, each of which assumes the one above it may be wrong.

Layer	Control	Implementation
Transport	Header hardening and origin control	Helmet with a restrictive CSP; CORS with an explicit allowlist — never a wildcard with credentials; a global <code>verifyOrigin</code> CSRF guard
Admission	Metering before parsing	Flood guard, per-group rate limiters, lockout check, and a Content-Length pre-check — all mounted before <code>express.json()</code>
Identity	Proof of possession	JWT (HS256, pinned) validated against a server-side session record; argon2id password verification; WebAuthn, TOTP, email OTP, recovery codes
Elevation	Step-up re-authentication	Possessing a session is not enough to change security settings. A 5-minute, session-bound JWT is traded for password plus current TOTP and carried in X-Step-Up, never in a URL
Authorisation	Owner-scoped access	Every <code>find</code> pairs the object id with <code>req.user._id</code> ; a miss returns 404 , never 403 , so the API never confirms that another user's object exists
Secrets	Encryption at rest	TOTP seeds and private keys sealed with AES-256-GCM; production refuses to boot without a real encryption key
Accountability	Append-only audit	A <code>SecurityEvent</code> log over a closed enum with a fixed metadata allowlist — no free-form field, so a secret cannot be smuggled into an event
Resilience	Recoverable failure	A global error boundary with one-shot self-heal for stale dynamic-import chunks, so a recoverable build skew never shows a white screen
Assurance	Boot-time configuration validation	Security-critical environment values are validated before serving; a nonsensical rate limit is warned about and ignored, so a typo can never disable protection

10.2 Authentication and Multi-Factor Authentication

Member sign-in is deliberately engineered against enumeration. Unknown-user and wrong-password produce identical bodies and comparable timing — the unknown-user path burns a throwaway password compare specifically to equalise it. Ban and inactive checks run **after** the password proof, because they were previously an unauthenticated oracle. Legacy bcrypt hashes are transparently upgraded to argon2id on successful login. Registration requires a server-issued single-use math CAPTCHA.

Factor	Implementation and Hardening
Email OTP	6 digits, 10-minute TTL, in-memory store, 5 wrong attempts before invalidation, 30-second resend cooldown, branded templates. The code is stored only after successful delivery , and is never returned in the JSON response
Authenticator app (TOTP)	speakeasy; 6 digits / 30 s / SHA-1 with a ± 1 -step window. The seed is encrypted at rest with AES-256-GCM. A replay cache keyed by account + secret + code makes a code single-use
Passkeys (WebAuthn)	@simplewebauthn/server — no hand-rolled crypto. The account is resolved from the verified credential only; registration challenges are bound to user + session + userHandle and spent before verification; signature-counter rollback is detected; a stable 32-byte <code>webauthnUserId</code> handle is maintained
Recovery codes	10 codes, roughly 49.5 bits each, generated with rejection sampling so there is no modulo bias. Only SHA-256 hashes are stored, the plaintext is shown exactly once, and single use is enforced by a conditional update on <code>usedAt: null</code>
Recovery email	6-digit verification with a 15-minute single-use proof. The address is not stored until verified, and it can never be the account's own address or another account's address
Step-up	A 5-minute, session-bound JWT traded for password + current TOTP. Setup, verify, disable, backup-code and recovery-email mutations all require it

- An account with `twoFactorEnabled` but no readable secret **falls back to email OTP** rather than locking the user out permanently.
- Disabling 2FA clears the seed entirely and destroys all recovery codes.
- Removing the last strong method is refused with `'409'` — an account cannot be stranded.
- An administrator whose stored credential was upgraded receives a **secret-free** notice (`'npm run notify-admins'`). The one-time credential hand-off (`'npm run send-admin-credentials'`) is kept as a **separate code path** precisely so the recurring channel can never grow a password field.

Password policy is defined once in `utils/passwordRules.js` and served over the wire so the client checklist cannot drift: 10–128 characters with upper case, lower case, digit and symbol; a blocklist of common passwords, decorated variants, sequences and repeated runs; and a refusal to accept a password built from the account's own email local part. A 0–4 strength score and a rule-by-rule checklist are returned to the UI.

Reset is a link flow — request → validate → complete — rather than a modal, because the link is followed from an email client, usually in a new tab and possibly hours later. The grant is 256 bits of randomness stored only as a SHA-256 hash with a Mongo TTL index and a 30-minute lifetime. Completion revokes **all** sessions, burns outstanding grants and emails the account owner. The request endpoint returns the same `200` whether or not the account exists or the mail went out.

10.3 Session Architecture

The session model is the most distinctive security decision in the project, because the two identity spaces are deliberately different.

Property	Member Sessions	Administrator Sessions
Time-based expiry	None. There is no idle timer and no <code>exp</code> claim	Yes — 10-minute idle window by default (<code>ADMIN_IDLE_MINUTES</code>)
Token lifetime	Not applicable	15 minutes by default (<code>ADMIN_TOKEN_LIFETIME_MINUTES</code>), must exceed the idle window or the server refuses to boot
Concurrency	One active session per account , enforced atomically through <code>User.currentSessionId</code> . A new sign-in revokes only the displaced session of the same account, so multi-account use is unaffected	Single session per administrator identity
Browser storage	Per-tab <code>sessionStorage</code> , with a shared account directory in <code>localStorage</code> and a <code>BroadcastChannel</code> relay	<code>localStorage</code> under a separate key; never visible to the member session layer
Renewal	Not applicable	Auto-renewal at roughly 60% of lifetime and on tab wake; any 401 or 403 clears the token and returns to <code>/admin/login</code>
Activity reporting	Not applicable	Client heartbeat at most once per minute keeps the server clock honest; a background-poll opt-out header prevents polling from extending an idle window

- Middleware validation is a fixed chain: session exists → owned → unrevoked → is the ****current**** session → not expired → account active.
- A 30-second validation cache keeps the hot path cheap, and it is invalidated synchronously on sign-out and on plan change, so “signed out” and “plan changed” take effect on the very next request.
- The bearer header is the only accepted location. Query-string tokens are rejected, and the JWT algorithm is pinned to HS256.
- ``Session.expiresAt`` exists purely as an operator lever; the application always writes ``null``.

DESIGN NOTE The multi-account design is unusual and worth explaining properly. Per-tab `sessionStorage` plus a shared directory means a user can hold three accounts open in three tabs and switch between them without signing out of any. It also means a cross-tab `storage` event must **not** sign the current tab out — `useAuth` ignores those events deliberately. Two tests (`adminSession`, `adminUserIsolation`) pin the ordering and the isolation rules.

10.4 Authorization Model

Middleware	Effect
<code>protect</code>	Validates the bearer JWT and the server-side session record; the security baseline for every private route
<code>adminOnly</code>	Requires an administrator identity; a user token is refused with 403
<code>userOnly</code>	Refuses administrator identities on member and provability routes — the admin console and the member app are genuinely separate worlds
<code>requireFeature(key)</code>	The entitlement gate; fails closed on unknown and prototype keys
<code>requireStepUp</code>	Requires a valid, unexpired, purpose-matched, session-bound step-up token
<code>streamTokenAuth</code>	Authenticates the Server-Sent Events subscription stream, which cannot set headers
<code>lockoutCheck</code>	Blocks a request from an IP or account currently inside the lockout ladder

Administrator sessions additionally carry two controls a member session does not: a mandatory `mustChangePassword` gate enforced server-side with a 403 until satisfied (with an exact method-and-path allowlist so the user can still reach the change-password page), and a server-defined idle window whose countdown is driven by the same configuration value in the client, so the two cannot disagree.

10.5 Rate Limiting and Body Protection

Control	Behaviour
Escalating lockout ladder	15 minutes → 1 hour → 6 hours → 24 hours, with the offence history decaying after 30 days and a separate three-strike account counter. The 2FA ladder is capped at 1 hour so an attacker cannot lock a victim out of their own recovery path
Credential-stuffing tripwire	20 distinct failed accounts from one IP in 15 minutes triggers a 5-minute cooldown — held in a bucket separate from the hard stop, so a shared NAT address is not permanently banned
Flood guard	A global per-IP envelope that meters traffic but never escalates the lockout ladder
Body budget	A global in-flight JSON byte cap. Exhausting it returns 503 with <code>Retry-After</code> , and an aborted upload releases its reservation
Oversized-body rejection	A <code>Content-Length</code> pre-check returning 413 before <code>express.json()</code> reads the socket
Configurable ceilings	Per-IP ceilings live in configuration because a security ceiling is also a capacity number: a deployment behind a shared NAT has every user behind one public address. Unset, empty or nonsensical values fall back to the documented default — a typo must never be able to switch rate limiting off

10.6 Secret Custody

- ****TOTP seeds**** cannot be hashed, because the server must run HMAC over them to verify a code. They are therefore encrypted with AES-256-GCM using a key from ``TOTP_ENCRYPTION_KEY``, with key-id envelopes and decrypt-only previous keys to support rotation (``TOTP_ENCRYPTION_KEY_PREVIOUS`, `kid=base64``). ``utils/totpSecret.js`` is the only module permitted to read or write a seed, including a lazy in-place migration from the legacy plaintext field.

Support for `TOTP_ENCRYPTION_KEY``

- ****Provability private keys**** are sealed in an AES-256-GCM envelope keyed from ``JWT_SECRET``, in exported PKCS#8 PEM form, retrievable only by the owner and stamped on export.
- ****Environment secrets**** are read by a loader that can re-read the file after boot — which is what makes the administrator “force re-check” of AI providers possible — while snapshotting OS-provided variable names first and reporting ****names only****, never values.
- ****Client-side**** password inputs block copy, cut, paste, the context menu and drag, and collapse any selection on focus.

10.7 Security Event Log

Each account has an append-only security history the member can read. The model enforces a closed enum of roughly 44 event types and a fixed metadata allowlist covering only `authMethod`, `factor`, `passkeyName`, `platform`, `mfaVerified` and `outcome`. There is **no free-form field**, which means a secret cannot be smuggled into an audit record through an error message. The `write()` helper never throws, so auditing can never break the operation it is auditing.

This is distinct from `usernotifications` and `adminevents`, which are **mutable** inbox surfaces. The audit log is append-only; the notification streams are not. Keeping those separate is what allows a member to dismiss a notification without erasing a security record.

10.8 Live Security Monitor and Audit

The administrator Security Center runs **45 live behavioural probes** — 25 core platform and 20 provability-layer — every 30 seconds, with a manual `?fresh=1` synchronisation. The probes execute the real attacker code path rather than checking configuration, so a passing probe is evidence of behaviour, not of intent.

- Per-probe status (Healthy, Warning, Critical, Error), framework chip, implementing file, latency, expandable detail rows and an expand-all control.
- Search, status chips with counts, a framework filter and summary tiles.
- A 20-second request timeout so the panel cannot hang on a stalled connection.
- Per-panel ``Promise.allSettled`` in the Overview tab, so one failing endpoint never blanks the rest.
- A complementary ****system detection and threat prediction**** feature reads those probe results and returns a severity, predicted risks and recommended actions. Its two most important properties: it cannot break detection (every failure path falls back to a rule-based verdict computed from the same probes), and model output is never trusted (severity clamped, categories fixed, strings bounded, malformed replies discarded whole).

A static audit record of **30 findings** — 2 critical, 8 high, 9 medium, 7 low, 4 informational — is maintained with 23 remediated in code and 7 open or accepted. The record lives in `SECURITY_AUDIT_REPORT.md` and is surfaced at `GET /api/admin/security`.

Chapter 11 · Testing and Quality Assurance

11.1 Test Strategy

The project uses Node's built-in test runner (`node --test`) rather than Jest or Vitest. This is a deliberate, stated preference for a dependency-light toolchain: 105 test files run with **zero test-framework dependencies**, which removes an entire category of supply-chain surface from a security-sensitive application.

Three test styles are used together, and each exists because of a specific constraint:

Style	What it covers	Example
Pure-logic unit tests	Isolated rules with no I/O — entitlement matrices, crypto primitives, proposal tallies, slot resolution, plan-day rollover, streak and wellness arithmetic	entitlements, web3, plan-day, wellness-missed-penalty, daily-schedule-slots
Real-HTTP integration tests	A server booted on a free port against a throwaway MongoDB, exercising the genuine middleware chain including rate limiting and CSRF	passkeys, recovery-codes, admin-session-expiry, password-reset-db
Source-level assertions	Wiring that <code>node --test</code> cannot import — JSX and CSS are read as text and matched with patterns	optimizeDepsInclude, planDayWiring, greetingWiring, navbarSticky, navbarMotion, navbarSurface, brandControl

Two further techniques are notable:

- Contract tests that fail on drift.** The frontend feature registry is tested against the server entitlement matrix; the client password checklist against the shipped server policy; and `optimizeDeps.include` against the dynamic imports that need it. If two copies of the truth diverge, the build fails.
- Adversarial probe suites.** `glitch-hunt.js` and `glitch-hunt-web3.js` attack a live API with authentication bypass, IDOR, injection, prototype-chain pollution, XSS, open redirect and mass-assignment probes, plus settlement and lost-update races. These are run by hand against a disposable instance and are deliberately excluded from CI because they write throwaway accounts to the application database.

11.2 Test Inventory

Scope	Files	Coverage
Server unit and integration	63	Roughly 658 tests (619 executed, 39 environment-gated skips when no test database is available)
Frontend unit and source-level	42	Roughly 244 tests
Live subscription flows	script	<code>test-subscription-flows.js</code> — the purchase, approval, cancellation and expiry flow end to end
Web3 end-to-end HTTP smoke	script	<code>test-web3-flows.js</code> — exercises all 20 provability features over HTTP
Adversarial probes	2 scripts	<code>glitch-hunt.js</code> , <code>glitch-hunt-web3.js</code>
Resilience stress	script	<code>test-ddos-resilience.js</code> — refuses to run against anything but loopback
Session and priority walks	2 scripts	<code>test-session-flows.js</code> , <code>test-priority-override-flows.js</code>
Native Android unit test	1	<code>MainActivityFilenameTest</code> — the download filename sanitiser
Static syntax gate	script	<code>npm run check</code> runs <code>node --check</code> across the entire security-critical server surface

Representative security-focused suites, each pinned to the specific failure it prevents:

- `auth-security` — login must never leak an OTP in JSON.
- `login-picture-footprint` — no image load on the sign-in path (a roughly 3 MB document outage).
- `totp-encryption` — round trip, key versioning, nonce freshness, tamper rejection, key rotation and lazy migration.
- `totp-replay-scope` — the one-time-use cache is scoped per account, not per seed.
- `recovery-code-generation` — rejection sampling, no modulo bias, no constant pad.
- `user-session-no-expiry` — the no-expiry policy and the paths that `*do*` end a session.
- `flood-guard` — 503 with `Retry-After`, 413 from `Content-Length`, reservation release on abort.
- `path-traversal` — a canary file outside the static root is never served.
- `support-chat-state` — a resolved thread is read-only for **both** sides.
- `subscription-panel-contract` — the administrator panel and the route actually agree on field names.
- `admin-users-projection` — the users grid projection supplies every field its readers use.

11.3 Continuous Integration Pipeline

`.github/workflows/ci.yml` runs on every push and pull request in three jobs plus a secret sweep. Its stated purpose is to keep the security audit report honest: the report's "0 vulnerabilities" claim had already silently expired because nothing ran `npm audit` on push — a nodemailer advisory landed after the audit was written and sat unnoticed until the workflow existed.

Job	Steps	Why it exists
server SuppliWise System Manual	<code>npm ci</code> → <code>npm run check</code> → <code>npm test</code> (with a real MongoDB service) → <code>npm audit --omit=dev</code>	A <code>SyntaxError</code> should never reach a review. The MongoDB service is required so the roughly 40 database suites execute rather than skip — a skipped test is not a passing test

Job	Steps	Why it exists
client	npm ci → eslint → npm test → vite build → npm audit	Warnings are allowed deliberately (a few known pre-existing hook warnings are not regressions); errors are not. A green build also proves the service worker is still generated
secrets	Credential-shaped pattern sweep across tracked docs and example config; .env tracked check	The remote is public, so a committed credential is a permanent breach. The sweep searches for structure , never a literal value — a real credential must not be pasted into the workflow file. It also warns that sanitising a file does not un-leak it, and rotation is required

NOTE glitch-hunt.js and the test-*-flows.js probes are deliberately **not** run in CI. They attack a live API and write throwaway accounts to the application database, so they require a running server and a disposable database. Document this decision rather than omitting it.

11.4 Coverage Matrix

Subsystem	Unit	Integration / HTTP	Notes
Authentication and sessions	Yes	Yes	Including a real WebAuthn authenticator and a real MongoDB
Multi-factor and recovery	Yes	Yes	Every factor has a full lifecycle including revocation and race conditions
Account security panels	Yes	Partial	Frontend panels covered by render and wiring tests
Entitlements and subscription	Yes	Yes	Plus live flow scripts
AI routing and providers	Yes	Partial	Wire formats, fallback chains, completion parsing, placeholder detection
Severity and priority	Yes	Partial	Escalate-only invariant, confidence bar, redaction, caching, fallbacks
Dashboard, intake, plan day	Yes	Yes	Daylight-saving transitions, leap days, dead hours, timezone handling
Wellness score and streaks	Yes	Yes	Missed-dose penalty windows, no NaN
Support chat	Yes	Yes	State machine plus a full HTTP suite
Notifications	Yes	Partial	Severe-flag backfill, dismissal, retention
Provability layer	Yes	Yes	Plus a 20-feature end-to-end HTTP smoke test
Provability plan gating	Yes	Yes	One path-keyed gate for the whole layer
Frontend presentation logic	Yes	No	Pure modules tested directly against hostile and partial payloads
Frontend wiring and CSS	Yes (source-level)	No	Read as text; see 11.1
Admin console	Yes	Yes	Projection, analytics shaping, security factor derivation, subscription panel contract

Chapter 12 · Deployment and Operations

12.1 Prerequisites

- Node.js **18** or higher — the server uses the built-in `fetch`, so no `node-fetch` is needed.
- MongoDB Community Server running locally, or MongoDB Atlas.
- An OpenRouter account for the default AI provider; Groq and Anthropic keys for the features routed to them.
- A Gmail account with 2FA enabled and an App Password, for OTP and reset email.
- **For Android builds only:** JDK 17, plus the Android SDK and Gradle installed by Capacitor.

12.2 Local Development Setup

Figure 3 — first run

```
git clone https://github.com/REIIdes/SuppliWise.git
cd SuppliWise

# Terminal 1 – backend
cd server
npm install
cp .env.example .env          # Windows: copy .env.example .env
# edit .env: JWT_SECRET, MONGO_URI, AI keys, SMTP
npm run dev                   # nodemon index.js

# Terminal 2 – frontend
cd my-react-app
npm install
npm run dev                   # Vite on http://localhost:5173
```

- **Two terminals are required** — the API and the client run as separate processes.
- The dev server proxies `/api` to `localhost:5000`, so the client needs no API configuration.
- For mobile testing on the same Wi-Fi, start the dev server with host exposure and open `http://:5173` on the phone. Set `ALLOW_LAN_ORIGINS=true` **in development only** if passkeys or secure cookies misbehave on a LAN origin.
- A fresh checkout contains no `.env`. It is gitignored by design and must never be committed.

12.3 Environment Configuration

`server/.env.example` is a 294-line commented reference. For every security-critical variable it documents not only what it is for but **what breaks without it**. The minimum required to boot locally is:

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/suppliwise
JWT_SECRET=<at least 32 characters, not the placeholder>

# AI providers – note the required _here suffix on placeholders:
# a placeholder written any other way is treated as a real credential
# and the admin panel shows it as live.
OPENROUTER_API_KEY=<your key>_here      # assessment, chat, polish, detail
GROQ_API_KEY=<your key>_here             # system detection & threat prediction
ANTHROPIC_API_KEY=<your key>_here        # priority assessment flagging
# Unset model overrides use the provider's pinned default.
OPENROUTER_MODEL=
GROQ_MODEL=
ANTHROPIC_MODEL=

# Email (OTP codes and reset links)
EMAIL_SERVICE=gmail
EMAIL_USER=<sender address>
EMAIL_PASSWORD=<16-character app password, no spaces>
EMAIL_FROM_NAME=SuppliWise
EMAIL_FROM_ADDRESS=<sender address>
```

Security-critical variables. Development has sensible fallbacks. In production the server refuses to start without them, because each failure mode is invisible until a real user is affected. Appendix C is the full reference; the four that most often cause a failed production boot are:

Variable	Why it matters
TOTP_ENCRYPTION_KEY	A TOTP seed cannot be hashed — the server must run HMAC over it to verify a code — so it must be encrypted at rest. Without a real key, anyone who can read the database can mint valid codes forever. Generate with <code>openssl rand -base64 32</code>
WEBAUTHN_RP_ID	A bare domain : no scheme, no port, no path. It must be the domain that actually serves the web app
WEBAUTHN_ORIGIN	Comma-separated <code>https</code> origins allowed to run a WebAuthn ceremony; also trusted by CORS
SuppliWise System Manual v1.0 WEB_ALLOWED_ORIGINS	Origins permitted to make credentialed requests. Never use a wildcard — <code>*</code> is incompatible with credentials

DEVELOPMENT-ONLY SWITCHES `ALLOW_LAN_ORIGINS` and `ALLOW_DEV_OTP_RESPONSE` must never be enabled in production, and the server refuses to boot if `ALLOW_DEV_OTP_RESPONSE` is `true` while `NODE_ENV=production`. The second returns OTP codes in the JSON response.

12.4 Production Readiness

- Set ``NODE_ENV=production`` explicitly. This is worth emphasising: the production guards are gated on it, so a deployment that omits it runs with every production check silently disabled.
- Provision real administrator accounts through ``ADMIN_ACCOUNTS`` (``alias|argon2id-hash|totp-secret`` triples, comma-joined) or the legacy single-admin ``ADMIN_ALIAS`` / ``ADMIN_PASSWORD_HASH`` / ``ADMIN_TOTP_SECRET`` form. ``ADMIN_EMAILS`` is a delivery-address mapping (``alias=address``), **not** a login credential; an unparseable entry is skipped rather than half-inserted.
- Terminate TLS in front of the application and set ``TRUST_PROXY=true`` **only** when a proxy you control sets ``X-Forwarded-For``.
- Serve the built client from ``my-react-app/dist`` with SPA history fallback, and set ``PUBLIC_WEB_URL`` to the address used in emailed links.
- Point ``MONGO_URI`` at a managed cluster with backups and point-in-time recovery.
- Run the syntax gate, the full test suite with a real test database, both ``npm audit`` scopes and the secret sweep before every release.

12.5 Android APK Build

Figure 4 — Capacitor build sequence

```
# 1. Build the web application
cd my-react-app
npm run build

# 2. Sync with Capacitor
npx cap sync android

# 3. Fix the Java version (required after every sync)
#   edit android/app/capacitor.build.gradle
#   sourceCompatibility JavaVersion.VERSION_21 -> VERSION_17

# 4. Build the APK
cd android
gradlew.bat assembleDebug      # macOS/Linux: ./gradlew assembleDebug

# 5. Output
#   android/app/build/outputs/apk/debug/SuppliWise.apk
```

IMPORTANT Step 3 is not optional and not a one-off. **After every ``npx cap sync``, change the Java source and target compatibility from ``VERSION_21`` back to ``VERSION_17`` in `android/app/capacitor.build.gradle`.** The sync regenerates that file, so a build that worked yesterday can fail today for a reason that has nothing to do with your changes.

Installation on a device: transfer `SuppliWise.apk` to the phone, enable “Install from Unknown Sources” in Android settings, tap the file, then launch SuppliWise from the app drawer.

12.6 Operational Tooling

Command	Purpose
<code>npm run dev (server)</code>	<code>nodemon index.js</code> with hot reload
<code>npm start (server)</code>	<code>node index.js</code> — does not set <code>`NODE_ENV=production`</code> , so set it explicitly in a real deployment
<code>npm test (both)</code>	The full suite: 63 server files, 42 frontend files
<code>npm run check (server)</code>	<code>node --check</code> across the security-critical source surface
<code>npm run lint (client)</code>	ESLint with warnings tolerated, errors fatal
<code>npm run test:flows (server)</code>	Live subscription flow checks
<code>node test-web3-flows.js</code>	End-to-end HTTP smoke across all 20 provability features
<code>npm run notify-admins</code>	Sends the secret-free notice that stored credentials changed. Safe to re-run
<code>npm run send-admin-credentials</code>	The one-time hand-off carrying alias, password and TOTP seed. Never scheduled. See the drift note in Section 6.4
<code>node glitch-hunt.js</code>	Adversarial probe suite against a running API
<code>node test-ddos-resilience.js</code>	Flood and DDoS resilience stress test (loopback only)
<code>npm run test:email</code>	SMTP configuration check
<code>docs/generate_system_manual.py</code>	Regenerates this manual from the source tree

Two design decisions here are worth defending: the recurring credential notice and the one-time credential hand-off are **two separate code paths** rather than one implementation with a flag, precisely so the recurring channel can never grow a password field. And `adminAccounts.js` can rewrite a spent hand-off password in `.env` while preserving the hash structure, so the file copy of a used credential cannot resurrect it.

Chapter 13 · Limitations and Future Work

13.1 Declared Limitations

A capstone panel rewards a team that knows precisely where its own system is weak. Present these as findings with positions taken, not as apologies.

#	Limitation	Position
L1	The provability layer is a simulated in-app chain. There is one process, one database and a mutex. No consensus, no replication, no second node, no difficulty adjustment and no fork-selection rule	Declared by design in the product documentation and in Chapter 9. Its value is demonstrable tamper evidence and a privacy-preserving anchor, not distributed consensus. The cryptography is real; the network is not claimed to be
L2	Block header hashes cover <code>prevHash</code> linkage and transaction hashes, not the human-readable transaction payload in <code>txs[].data</code>	Each transaction's own <code>txHash</code> authenticates its payload, and re-verification recomputes it. Folding the payload into the header is a small, well-scoped change identified as future work
L3	Payments are manual. Checkout is a proof-of-payment submission verified by an administrator; there is no payment gateway	A deliberate scope decision (Section 1.3). Pricing is server-computed and snapshotted onto each request so a price change cannot alter an existing request
L4	The system is advisory, not a medical device. It does not diagnose, and the output is not a substitute for professional care	Enforced at four points: prompt rules, output sanitisation, client disclaimer on every card, and a consult-a-professional notice in every generated PDF
L5	Priority flagging is a rule engine plus an AI second opinion. A fixed list only finds what someone anticipated	The AI layer is escalate-only by construction, so it can add coverage without ever adding a false clearance. Deliberately conservative — a spurious flag blocks a member's assessments
L6	Chats and the AI assistant are not crisis-detection channels, and the support channel requires an account	When recreational drug use is reported, the generated plan surfaces a dedicated "Seeking Support" section with four Philippine resources (DOH SAH 1550, DDB, DSWD Yakap Bayan, NCMH 1553). Named as the highest-priority future enhancement
L7	The AI layer depends on third-party providers whose model identifiers and credit state can change without notice	Mitigated by the routing table, the live provider probe that distinguishes "Reachable" from "last call succeeded", the <code>chat</code> fallback, and the deterministic rule engine that requires no network at all
L8	Model ids are key-specific and were selected against the shipped keys	Documented in the AI chapter with the warning that a model a key cannot serve fails the first real call, not the health check

13.2 Future Enhancements

- **Clinical-grade citation verification.** Machine-check every PMID and DOI against NCBI and Crossref at generation time, rejecting any citation that does not resolve. This is the single highest-value improvement, because an unverified citation in a health tool is worse than no citation.
- **Crisis and red-flag detection in the chat assistant.** Extend the escalate-only severity model to free-form conversation, with immediate resource surfacing.
- **A payment gateway** with webhook settlement, replacing manual proof-of-payment while keeping the administrator override path for edge cases.
- **Transaction-payload hashing in the block header**, closing limitation L2.
- **Consensus and replication** for the provability layer, or an explicit migration to an existing external chain, removing limitation L1.
- **Clinician accounts** with scoped, consent-gated access to shared profiles, replacing the account-free share link.
- **Localisation.** The support resources, currency catalogue and prompt language are Philippine- and English-specific; a resource-per-country model and a multilingual prompt would extend reach.
- **Coverage measurement.** A dependency-light coverage reporter, integrated into CI, would make the quality claim quantitative rather than enumerated.
- **Push notifications** for the daily supplement reminder and for support replies, via the existing PWA.
- **Two-factor recovery hardware** — security keys as a distinct passkey tier, and verifiable credential-based account recovery.

Chapter 14 · Team and Contribution Record

14.1 Team Roster

Member	Git Author	Role	Primary Contribution
[Member 1]	REIIides	[Team Lead / Backend & Provability]	[Fill in — e.g. ledger and engine, DAO, market escrow, data model]
[Member 2]	KlaporeDevs	[Frontend & Mobile]	[Fill in — e.g. dashboard, assessment wizard, PWA and Android packaging]
[Member 3]	Suo-Cyber	[AI Subsystem & Security]	[Fill in — e.g. AI routing, provider probes, priority flagging, chat safety]
[Member 4]	maweteh	[UI, Testing & Documentation]	[Fill in — e.g. admin console, test suites, manuals and CI]

The git author names above are taken verbatim from the repository's commit history. Replace the bracketed role and contribution columns with the team's actual allocation before submission.

Metric	Value
Contributors (by commit authorship)	4
Total commits	102
Development window	2026-05-27 to 2026-10-02
Modules in active development	JDMv1 through JDMv6 (six iterations)
Documentation files authored	118 markdown documents plus the docs/ manual set

Suggested per-member templates for the individual chapters each student submits:

- ****Introduction / Objectives**** — problem framing, research survey, and the specific problem you personally owned.
- ****Architecture**** — the layers you designed and why, with the trade-offs you rejected.
- ****Subsystem chapters**** — your module's requirements, data structures, algorithms and the specific defects you found and fixed.
- ****Testing**** — the suites you authored and the regressions each one now prevents.
- ****Conclusion and future work**** — an honest evaluation of your own module against its requirements.

14.2 Module Ownership

Module	Primary Files	Owner
Authentication and account security	server/routes/auth.js, passkeys.js, totp.js, recoveryCodes.js, security.js, passwordReset.js, middleware/auth.js, middleware/stepUp.js	[Member]
Provability layer	server/blockchain/*, server/routes/web3/*, server/models/Web3.js	[Member]
AI subsystem	server/utils/aiRouter.js, aiProviders.js, priorityFlagging.js, severity.js, chatSafety.js, systemDetection.js, server/routes/recommend.js, chat.js	[Member]
Subscription and entitlements	server/utils/entitlements.js, subscriptionState.js, subscriptionBus.js, planCatalogue.js, currency.js, server/routes/subscription.js	[Member]
Tracking and adherence	server/routes/dashboard.js, utils/planDay.js, wellnessScore.js, intakeWindows.js, dailyScheduleSlots.js, my-react-app/src/Pages/TrackIntakePage.jsx	[Member]
Admin console	server/routes/admin.js, adminSupportChats.js, utils/attack_probes.js, utils/securityAudit.js, my-react-app/src/Pages/AdminDashboard.jsx, Components/SecurityStatus	[Member]
Assessment and results UI	my-react-app/src/Pages/AssessmentPage.jsx, ResultsPage.jsx, HistoryPage.jsx, RecommendationsPage.jsx	[Member]
Mobile and packaging	my-react-app/vite.config.js, capacitor.config.json, android/, generate-android-icons.cjs	[Member]
Testing and CI	server/Test File/*, my-react-app/src/**/*test.js, .github/workflows/ci.yml	[Member]
Documentation	README.md, SYSTEM_ARCHITECTURE.md, BLOCKCHAIN_FEATURES.md, SECURITY_AUDIT_REPORT.md, docs/*	[Member]

Appendix A · API Endpoint Reference

Complete endpoint listing, grouped by mount point. Guard notation: **P** = protect (member session), **A** = adminOnly, **SU** = requireStepUp, **F** = requireFeature, **—** = unauthenticated. The complete listing is 220 endpoints across 22 route groups plus 9 Web3 sub-routers.

A.1 Authentication — /api/auth

Method	Path	Guard	Purpose
GET	/captcha	—	Issue a server-side single-use maths CAPTCHA
POST	/register	—	Create an account (argon2id hash, name validation, CAPTCHA)
POST	/login	—	Password step; issues an MFA transaction when 2FA is on
POST	/admin-login	—	Administrator alias and password step
POST	/verify-admin-2fa	—	Administrator TOTP step → administrator JWT
POST	/admin-refresh	—	Renew an administrator token before expiry
POST	/verify-login-otp	—	Email-OTP second factor → member session
POST	/login-2fa	—	Authenticator second factor → member session
POST	/remember	—	Mint the opt-in “save my login” credential
POST	/resend-login-otp	—	Re-send the login OTP
POST	/setup-2fa	P SU	Begin TOTP enrolment; returns a QR code and secret
POST	/verify-2fa	P SU	Confirm enrolment and enable 2FA
POST	/disable-2fa	P SU	Disable the second factor and destroy the seed
POST	/request-email-otp	—	Email 2FA enrolment OTP
POST	/verify-email-otp	—	Confirm email 2FA enrolment
POST	/two-factor-method	P	Switch between authenticator and email codes
POST	/forgot-password	—	Start the emailed-link reset (always 200)
POST	/resend-password-reset-otp	—	Re-send the reset code
POST	/verify-password-reset-otp	—	Verify the 6-digit reset code
POST	/reset-password	—	Complete the reset and revoke all sessions
GET	/me	P	Current user, subscription and plan/feature flags
POST	/logout	P	Revoke this session
POST	/change-password	P	Change password (requires the current one)
POST	/sign-out-all	P	Revoke every other session
PUT	/profile	P	Update name, date of birth, gender and pictures
POST	/admin-activity	P	Administrator heartbeat beacon

A.2 Account Security

Method	Path	Guard	Purpose
POST	/api/security/backup-codes/redeem	—	Public pre-session recovery-code sign-in (registered before the router's protect)
POST	/api/security/step-up	P	Issue a 5-minute step-up token
GET	/api/security/summary	P	Security posture summary
GET	/api/security/devices	P	Signed-in devices
POST	/api/security/devices/:id/revoke	P SU	Revoke one device
POST	/api/security/devices/revoke-others	P SU	Revoke all other devices
GET	/api/security/events	P	This account's security event history
POST	/api/security/backup-codes/generate	P SU	Mint recovery codes
DELETE	/api/security/backup-codes	P SU	Invalidate all recovery codes
POST	/api/security/recovery-email/request	P SU	Send recovery-email verification
POST	/api/security/recovery-email/verify	P	Confirm the recovery email
DELETE	/api/security/recovery-email	P SU	Remove the recovery email
GET	/api/auth/passkeys	P	List the account's passkeys
POST	/api/auth/passkeys/register/option	P SU	WebAuthn creation options
POST	/api/auth/passkeys/register/verify	P SU	Verify and store a credential

Method	Path	Guard	Purpose
PATCH	/api/auth/passkeys/:id	P	Rename a passkey
DELETE	/api/auth/passkeys/:id	P SU	Hard-delete a credential
POST	/api/auth/passkeys/login/options	—	Passwordless challenge options
POST	/api/auth/passkeys/login/verify	—	Passwordless assertion → session
POST	/api/auth/totp/setup	P SU	Begin TOTP enrolment
POST	/api/auth/totp/verify-setup	P SU	Confirm and enable
POST	/api/auth/totp/disable	P SU	Disable the authenticator
POST	/api/auth/totp/verify	—	Verify a code at sign-in or redemption
POST	/api/auth/recovery-codes/generate	P SU	Mint a set (plaintext once)
POST	/api/auth/recovery-codes/regenerate	P SU	Replace the previous batch
POST	/api/auth/recovery-codes/verify	—	Test or redeem one code
GET	/api/auth/sessions	P	Signed-in device list
DELETE	/api/auth/sessions/:sessionId	P	Revoke one device
POST	/api/auth/sessions/revoke-others	P	Revoke all other devices
POST	/api/auth/sessions/logout-all	P	Revoke every session
GET	/api/auth/password-reset/rules	—	Public password-policy rules
POST	/api/auth/password-reset/request	—	Issue a reset grant and email
GET	/api/auth/password-reset/validate	—	Is this reset link still usable?
POST	/api/auth/password-reset/complete	—	Set the new password; revoke all
POST	/api/auth/password-reset/recovery-code	—	Redeem a recovery code
POST	/api/auth/password-reset/authenticator	—	Redeem an authenticator code
POST	/api/auth/password-reset/passkey/options	—	WebAuthn options for reset
POST	/api/auth/password-reset/passkey/verify	—	Complete the passkey proof
POST	/api/auth/password-reset/recovery-email	—	Send a recovery-email code

A.3 Member Domain

Method	Path	Guard	Purpose
POST	/api/assessment	P	Submit an assessment (severity detection, priority flagging, expiry)
GET	/api/assessment/history	P	Paginated assessment history
GET	/api/assessment/me	P	The member's latest assessment
GET	/api/assessment/priority-status	P	Is a Priority review open?
GET	/api/assessment/results/:assessmentId	P	Stored AI results
GET	/api/assessment/user/:userId	P A	Administrator read of a member assessment
PATCH	/api/assessment/:id/results	P	Save the AI result snapshot
PATCH	/api/assessment/:id/priority	P A	Raise or release a Priority review
DELETE	/api/assessment/:id	P	Delete an assessment and orphaned rows
POST	/api/recommend	P	Generate recommendations (AI with clinical fallback)
POST	/api/supplement-detail	P	Personalised supplement deep dive
POST	/api/polish	P	Rewrite free-text notes into clinical language
POST	/api/chat	P F	AI chat assistant (Ultimate entitlement)
GET	/api/dashboard	P	Today's plan by slot plus the wellness score
POST	/api/dashboard/intake	P	Tick one dose
POST	/api/dashboard/intake/bulk	P	Tick a whole slot in one call
POST	/api/dashboard/energy	P	Record the energy level
POST	/api/dashboard/add-supplement	P	Add a supplement to the plan
POST	/api/dashboard/remove-supplement	P	Remove a supplement
POST	/api/dashboard/reset	P	Reset tracking
GET	/api/dashboard/day/:dayKey	P	One plan day

Method	Path	Guard	Purpose
GET	/api/dashboard/calendar/:year/:month	P	Month grid
GET	/api/dashboard/weekly-adherence	P	Seven-day adherence
GET	/api/dashboard/my-plan	P	The full plan, grouped by slot
GET	/api/dashboard/current-supplements	P	Is this member already on a plan?
GET	/api/insights	P F	Insights and analytics
GET	/api/notifications	P	Inbox with severe-flag backfill
PATCH	/api/notifications/:id/read	P	Mark one read
POST	/api/notifications/read-all	P	Mark all read
POST	/api/notifications/delete-read	P	Delete read rows
DELETE	/api/notifications/:id	P	Dismiss one (soft, idempotent)
GET	/api/support-chat	P	The member's threads
POST	/api/support-chat	P	Start a thread with a category and subject
GET	/api/support-chat/:id	P	Thread and transcript
POST	/api/support-chat/:id/messages	P	Reply (author from the session)

A.4 Subscription

Method	Path	Guard	Purpose
GET	/api/subscription/plans	—	Public plan catalogue and payment instructions
POST	/api/subscription/purchase	P	Self-serve grant (warned at boot when enabled)
POST	/api/subscription/requests	P	Submit a plan request with proof of payment
GET	/api/subscription/requests	P	“My requests” (never returns the image)
GET	/api/subscription/requests/:id	P	One request
POST	/api/subscription/downgrade	P	Immediate plan removal
POST	/api/subscription/cancel-requests	P	Request a reviewed cancellation
GET	/api/subscription/cancel-requests	P	“My cancellations”
GET	/api/subscription	P	Effective subscription (paid + override)
GET	/api/subscription/feature/:key	P	A single entitlement check
GET	/api/subscription/stream	P*	SSE plan-change stream (streamTokenAuth)

A.5 Administrator

Method	Path	Guard	Purpose
GET	/api/admin/overview	P A	Dashboard analytics: facets, trends, runway, activation
GET	/api/admin/users	P A	Users grid with subscription and security projection
PATCH	/api/admin/users/:id/account	P A	Ban, unban, role or status change
DELETE	/api/admin/users/:id/lockout	P A	Clear an account lockout
DELETE	/api/admin/users/:id	P A	Delete the account and cascade
GET	/api/admin/admins	P A	List administrator accounts
PATCH	/api/admin/admins/:id	P A	Enable, disable or rename an administrator
DELETE	/api/admin/admins/:id/lockout	P A	Clear an administrator lockout
GET	/api/admin/admins/credential-notice/preview	P A	Preview the secret-free recurring notice
POST	/api/admin/admins/notify-credentials	P A	Send the recurring notice
GET	/api/admin/admins/credential-handoff/preview	P A	Preview the one-time hand-off
POST	/api/admin/admins/credential-handoff	P A	Send the one-time hand-off (carries live secrets; has a cooldown)
GET	/api/admin/users/:id/subscription	P A	Read the subscription record
POST	/api/admin/users/:id/subscription	P A	Grant or set the paid layer
PATCH	/api/admin/users/:id/subscription	P A	Apply an override or restore the original
GET	/api/admin/subscription-requests	P A	Purchase-request queue
GET	/api/admin/subscription-requests/:id	P A	One request with the full receipt
POST	/api/admin/subscription-requests/:id/approve	P A	Approve → setPaid

Method	Path	Guard	Purpose
POST	/api/admin/subscription-requests/:id/reject	P A	Reject with a note
GET	/api/admin/subscription-cancel-requests	P A	Cancellation queue
POST	/api/admin/subscription-cancel-requests/:id/approve	P A	Approve → remove
POST	/api/admin/subscription-cancel-requests/:id/reject	P A	Reject
GET	/api/admin/security/monitor	P A	Live 45-probe monitor (?fresh=1)
GET	/api/admin/security	P A	Static security audit record
GET	/api/admin/ai	P A	Provider panel and the routing description
POST	/api/admin/ai/check	P A	Force a provider health probe
GET	/api/admin/profile	P A	Own administrator profile (pictures excluded)
GET	/api/admin/profile/pictures	P A	Administrator pictures via a slow-tolerant connection
PATCH	/api/admin/profile	P A	Update the administrator profile
GET	/api/admin/session-status	P A	Idle and lifetime status
PATCH	/api/admin/profile/password	P A	Change own password (allowed while mustChangePassword)
POST	/api/admin/profile/authenticator/rotate	P A	Rotate the administrator TOTP secret
GET	/api/admin/notifications	P A	Administrator event bell
POST	/api/admin/notifications/read	P A	Mark events read
POST	/api/admin/notifications/read-all	P A	Mark all read
POST	/api/admin/notifications/delete-read	P A	Delete read events
DELETE	/api/admin/notifications/:id	P A	Delete one event
GET	/api/admin/chats	P A	Support queue ordered by status and recency
GET	/api/admin/chats/next-id	P A	Next thread id
GET	/api/admin/chats/:id	P A	Thread and transcript
POST	/api/admin/chats/:id/messages	P A	Reply (2,000-character cap)
POST	/api/admin/chats/:id/resolve	P A	Resolve the thread
POST	/api/admin/chats/:id/reopen	P A	Reopen the thread
POST	/api/admin/chats/:id/assign	P A	Assign to an alias
DELETE	/api/admin/chats/:id	P A	Delete the conversation
GET	/api/health	—	Liveness probe

A.6 Provability (/api/web3)

Method	Path	Guard	Purpose
GET	/verify/:code, /verify/:code/qr	—	Public supply-chain verification, recomputing every proof
GET	/share/:token	—	Public read-only shared health profile
GET	/wallet	P U	Own wallet, DID, balance and stake
GET	/wallet/private-key	P U	Export the signing key (stamps the export time)
GET	/chain	P U	Block explorer
GET	/chain/verify	P U	Incremental chain integrity verification
GET	/tx/:hash	P U	Transaction lookup by hash
GET	/config	P U	DAO-governed parameters
GET/POST	/supply/batches	P U	List or create supply batches
POST	/supply/batches/:id/events	P U	Append a journey step
POST	/supply/batches/:id/certifications	P U	Attach a certification digest
GET	/rewards/status	P U	Balance, streak and achievements
POST	/rewards/checkin	P U	Daily healthy-habit check-in
POST	/rewards/intake	P U	Reward for a tracked intake day
POST	/rewards/assessment	P U	Reward for a completed assessment
GET	/rewards/events	P U	Reward ledger
GET	/rewards/nfts	P U	Owned achievement NFTs
POST	/rewards/achievements/check	P U	Mint newly earned achievements

Method	Path	Guard	Purpose
POST	/stake, /unstake	P U	Stake or unstake WELL
GET	/loyalty	P U	Loyalty codes
POST	/loyalty/redeem	P U	Redeem a loyalty code against an order
GET/POST	/market/listings	P U	Browse or create listings
POST	/market/orders	P U	Buy → escrow
GET	/market/orders	P U	Own orders
POST	/market/orders/:id/confirm	P U	Confirm delivery → release escrow
POST	/market/orders/:id/dispute	P U	Open a dispute
GET	/market/disputes	P U	Dispute list
POST	/market/disputes/:id/vote	P U	Cast a juror vote
GET	/dao/config	P U	Current parameters
GET/POST	/dao/proposals	P U	List or create proposals
POST	/dao/proposals/:id/vote	P U	Vote with held-plus-staked weight
GET/POST	/knowledge	P U	Knowledge base listing or publishing
POST	/knowledge/:id/upvote	P U	Upvote (rewards author and curator)
GET	/health/ledger	P U	Anchored health ledger
POST	/health/ledger/anchor	P U	Anchor a snapshot
GET	/health/export	P U	Export health data
POST	/health/backup	P U	Create an encrypted backup
GET/POST	/data/shares	P U	Research data consents
DELETE	/data/shares/:id	P U	Revoke consent and destroy the payload
POST	/storage/pin	P U	Pin encrypted ciphertext (content address)
GET	/storage/:cid	P U	Read an owned pinned object
POST	/recommendations/anchor	P U	Anchor an AI-proof of a recommendation
GET	/recommendations/anchor/:assessmentId	P U	Verify the anchor
GET/POST	/profile-shares	P U	Own share links; create a revocable link
DELETE	/profile-shares/:id	P U	Revoke a share link
GET	/trials	P U	Clinical trials
POST	/trials/:id/optin	P U	Opt in (rewarded)
POST	/trials/:id/withdraw	P U	Withdraw consent
GET	/oracle/feeds	P U	Oracle price, research and market feeds
POST	/oracle/refresh	P U	Refresh oracle values
GET	/experts	P U	Professionals
POST	/experts/:id/book	P U	Book and pay from the wallet
GET	/bookings	P U	Own bookings
POST	/bookings/:id/cancel	P U	Cancel and refund

Guard key for A.6 — P = member session, U = userOnly (administrator identities refused), and every provability path additionally passes a path-keyed plan gate (web3, market or dao).

Appendix B · Database Collection Reference

MongoDB with Mongoose 8. Thirty-three core collections plus 25 provability collections. No migrations are required because the AI result snapshot is a schemaless `Mixed` field.

Collection	Key fields	Indexes / constraints
users	firstName, lastName, unique email, argon2id password, dateOfBirth, gender, profilePicture, bannerPicture, timeZone, twoFactorEnabled, twoFactorMethod, twoFactorSecretEnc (select:false), webauthnUserId (select:false), recoveryEmail, subscriptionRecord.{paid,override,history}, lastLoginAt/ip/UserAgent/Location, accountRole, accountStatus, currentSessionId, sessionVersion	{createdAt:-1}, {lastLoginAt:-1}, {subscriptionPlan:1, subscriptionExpiresAt:1}; post-delete hooks cascade to 7 dependent collections
assessments	user, the full four-step intake, aiResults (<code>Mixed</code> , includes wellnessBaseline), expiresAt (default +5 calendar years), priority, flagReasons, flaggedAt, resolvedAt, userEmail, userName snapshot	{user:1, createdAt:-1}
intakerecords	user, assessment, supplementName, dosage, priority, scheduledTime, timeSlot, taken, takenAt, date, dayKey	{user:1, assessment:1, dayKey:1}, {user:1, dayKey:1}, date, dayKey
dashboardmetrics	user, assessment, currentStreak, longestStreak, totalDaysTracked, overallAdherence, lastTrackedDate, lastCompletedDay, streakAwardedToday, wellnessScore, energyLevel, assessmentStartDate, isActive	{user:1, assessment:1}, {user:1, isActive:1}
sessions	<code>_id</code> is the session id (the JWT <code>sid</code>); user, tokenHash, rememberHash, createdAt, lastActivityAt, deviceLabel, platform, ip, location, trustedAt, authMethod, mfaVerified, passkeyId, expiresAt, revokedAt	{user:1, createdAt:-1}, {user:1, revokedAt:1}, rememberHash
passkeys	user, globally unique credentialId, publicKey, counter, transports, deviceType, backedUp, aaguid, attestationFormat, name, userVerified, lastUsedAt, signInCount	{user:1, createdAt:-1}; credentialId unique
authchallenges	unique base64url challenge, flow (register/login/reset), user, sid, userHandle, attempts, maxAttempts, consumedAt	TTL index on expiresAt
mfatransactions	unique tokenHash, user, methods[], attempts, maxAttempts (10), primaryMethod, ip, userAgent, location, consumedAt, consumedBy	{user:1, consumedAt:1, expiresAt:1}; TTL index on expiresAt
backupcodes	user, batchId, sha256 codeHash, usedAt	{user:1, batchId:1}, {user:1, usedAt:1}
passwordresettokens	user, unique tokenHash (select:false), codeHash (select:false), codeAttempts, expiresAt, usedAt, requestIp, userAgent	TTL index on expiresAt; single-use conditional claim
securityevents	user, type (44-value enum), success, ip, userAgent, location, reason (≤ 160), meta (closed allowlist : authMethod, factor, passkeyName, platform, mfaVerified, outcome)	{user:1, createdAt:-1}, {createdAt:1}; append-only
usernotifications	user, type, title, detail, assessmentId, read, dismissed	{user:1, dismissed:1, createdAt:-1}, {user:1, read:1, dismissed:1, createdAt:-1}
adminevents	type enum (new-device-login, security, account, severe-flag, resolved, subscription, subscription-request, subscription-cancel-request, support-chat), title, detail, user, readBy[], assessmentId, linkUserId	{createdAt:-1}, {type:1, createdAt:-1}, {user:1, createdAt:-1}
subscriptionrequests	user, plan, months (1–12), server-priced snapshot (currency, symbol, amountPhp, amount, formattedAmount), reference, note, proof (base64 data URL), proofMime, proofBytes, status, review.{by,at,note}, grantedPlan, grantedDays	{status:1, createdAt:-1}, {user:1, createdAt:-1}
subscriptioncancelrequests	user, plan, mode (immediate/review), reason, status (pending/applied/rejected), review.{by,at,note}, appliedPlan, appliedAt	{status:1, createdAt:-1}, {user:1, createdAt:-1}
chatthreads	user, subject, category, status, denormalised lastMessageAt / Preview / By, messageCount, unreadByAdmin, unreadByUser, assignedTo, resolved	{status:1, lastMessageAt:-1}, {user:1, lastMessageAt:-1}, {unreadByAdmin:-1, lastMessageAt:-1}
chatmessages	thread, user, author (from the session, never the body), authorName, body (≤ 2000)	{thread:1, createdAt:1}; append-only
adminaccounts	unique alias, passwordHash (select:false), totpSecret (select:false), enabled, profilePicture, bannerPicture, picturesUpdatedAt, lastLoginAt, lastActivityAt, mustChangePassword, passwordChangedAt	alias unique
supplementdetails	unique nameKey, name, detail (<code>Mixed</code>), variants (<code>Mixed</code> map of profile digest → detail)	nameKey unique and indexed; variants declared explicitly because strict mode would otherwise strip it

Collection	Key fields	Indexes / constraints
swblocks	unique index, timestamp, prevHash, nonce, hash, txs[] (txHash, type, actor, data, dataHash)	{txs.txHash:1}
swaudits	key ('chain'), upTo, prevHash, verifiedAt, checked — the last proven-good chain position	key unique
swwallets	user (unique, sparse), did, unique address, publicKey, privateKeyEnc {iv, ct, tag}, label, isSystem, balance, staked, earnedTotal, spentTotal, lastStakeAccrualAt, welcomeBonusAt, keyExportedAt	user unique+sparse, address unique
swconfigs	key ('main'), params (Mixed, defaults DEFAULT_PARAMS), updatedBy, updatedAt	key unique
swsupplybatches	unique code, productName, brand, notes, createdBy, events[] (7 forward steps), certifications[] (lab/organic/non-gmo/third-party/gmp/other)	code unique
swlistings	seller, sellerUser, title, description, brand, category, priceWell, stock, active	—
sworders	listing, listingTitle, buyer, buyerAddress, sellerAddress, sellerUser, qty, unitPrice, total, fee, discountCode, discount, status, escrowTx, settleTx, settledAt	status indexed
swdisputes	order (unique), opener, reason, jurors[], votes[] (juror, choice, weight, at), status, outcome, resolvedTx	order unique; status indexed
swproposals	title, description, param, value, proposer, proposerAddress, status (active/passed/rejected), votes[], endsAt, executedTx	status indexed
swknowledgeposts	author, authorAddress, type, title, body, upvoters[], upvotes, rewardPaid, txHash	—
swrewardevents	user, kind, refId, day, amount, txHash, at	unique {user:1, kind:1, refId:1} (idempotency), {user:1, kind:1, day:-1}
swnfts	owner, ownerAddress, unique tokenId, kind, name, description, imageSeed, serial, mintedAt, txHash	tokenId unique; unique {owner:1, kind:1}
swloyaltycodes	unique code, owner, valueWell, status, redeemedBy, redeemedOrder	code unique; status indexed
swdatashares	user, scope, recipient, datasetCid, reward, rewardTx, status, consentTx, revokeTx, revokedAt	status indexed
swstorageobjects	cid, owner, kind, ciphertext, iv, tag, size, pinnedAt	unique {cid:1, owner:1} — deliberately not unique on cid alone
swhealthanchors	user, digest, assessmentCount, intakeCount, latestAssessmentAt, blockIndex, txHash	—
swsharelinks	unique token, user, expiresAt, revoked, views	token unique
swrecanchors	user, assessmentId, logicVersion, inputHash, outputHash, combinedHash, blockIndex, txHash	unique {user:1, assessmentId:1}
swtrials	title, sponsor, phase, description, rewardWell, spots, status	status indexed
swtrialconsents	user, trial, status (opted-in/withdrawn), reward, rewardTx, termsHash, consentTx, withdrawTx, withdrawnAt	status indexed; unique {user:1, trial:1}
sworaclefeeds	unique key, label, value, unit, category, source, txHash, updatedAt	key unique
swexperts	name, specialty, title, credential, bio, rateWell, address, active	—
swbookings	expert, user, userAddress, hours, cost, status, txHash	—

Appendix C · Environment Variable Reference

Grouped by purpose. Variables marked **production-required** cause the server to refuse to boot when absent or misconfigured. Full commentary is in `server/.env.example` (294 lines).

Group	Variable	Default / note
Core	PORT	5000
Core	MONGO_URI	Required. Absence exits with code 1
Core	JWT_SECRET	Production-required. ≥32 characters and not the documented placeholder
Core	NODE_ENV	Set to <code>production</code> explicitly in a real deployment — every production guard is gated on it
Core	PUBLIC_WEB_URL	Production-required. The origin used in emailed links
AI	OPENROUTER_API_KEY	Required for AI features. Placeholders must end in <code>_here</code> , or the admin panel will show them as live credentials
AI	GROQ_API_KEY	System detection and threat prediction
AI	ANTHROPIC_API_KEY	Priority assessment flagging and chat
AI	OPENAI_API_KEY	Optional alternative provider in the registry
AI	LLAMA_API_KEY, LLAMA_BASE_URL, LLAMA_MODEL	Optional self-hosted OpenAI-compatible endpoint
AI	OPENROUTER_MODEL	Unset → the provider's pinned default
AI	GROQ_MODEL, ANTHROPIC_MODEL	Unset → pinned default. Must be an id your key can reach, and the account must have credit
AI	ALLOW_DEV_OTP_RESPONSE	Development only. Returns OTP codes in JSON; production refuses to boot if this is <code>true</code>
Email	EMAIL_SERVICE	SMTP transport, e.g. <code>gmail</code>
Email	EMAIL_USER, EMAIL_PASSWORD	Sending account and its 16-character app password (no spaces)
Email	EMAIL_FROM_NAME, EMAIL_FROM_ADDRESS	Branding and envelope sender
Admin	ADMIN_ALIAS	Single-administrator alias (legacy form)
Admin	ADMIN_PASSWORD_HASH	Single-administrator hash (legacy form)
Admin	ADMIN_TOTP_SECRET	Single-administrator TOTP seed (legacy form)
Admin	ADMIN_ACCOUNTS	Multi-administrator form: comma-joined <code>alias argon2id-hash totp-secret</code> triples
Admin	ADMIN_EMAILS	Comma-joined <code>alias=address</code> pairs. A delivery address, not a login credential ; unparseable entries are skipped
Auth security	TOTP_ENCRYPTION_KEY	Production-required. <code>openssl rand -base64 32</code> . Production also rejects a JWT-derived fallback
Auth security	TOTP_ENCRYPTION_KEY_PREVIOUS	<code>kid=base64</code> rotation; decrypt-only
Auth security	WEBAUTHN_RP_NAME	Relying-party display name
Auth security	WEBAUTHN_RP_ID	Production-required. A bare domain: no scheme, no port, no path
Auth security	WEBAUTHN_ORIGIN	Production-required. Comma-separated <code>https</code> origins; also trusted by CORS
Auth security	WEBAUTHN_TIMEOUT_MS	Ceremony timeout
Auth security	WEB_ALLOWED_ORIGINS	Origins permitted to make credentialed requests. Never a wildcard
Auth security	ALLOW_LAN_ORIGINS	Development only. Trusts <code>http://192.168.x.x:port</code> for on-site phone testing
Sessions	ADMIN_IDLE_MINUTES	10. Whole number 1–1440; drives both the client countdown and the server check
Sessions	ADMIN_TOKEN_LIFETIME_MINUTES	15. Must exceed <code>ADMIN_IDLE_MINUTES</code> or the server refuses to boot
Rate limits	AUTH_RATE_LIMIT_MAX	Default 20 per 15 minutes (400 on loopback)
Rate limits	AUTH_RATE_LIMIT_MAX_LOCAL	Loopback ceiling
Rate limits	AUTH_SENSITIVE_RATE_LIMIT_MAX	Ceiling for the 12 credential paths
Rate limits	SECURITY_RATE_LIMIT_MAX	Ceiling for the security-center probes
Infrastructure	TRUST_PROXY	Only set <code>true</code> when a proxy you control sets <code>X-Forwarded-For</code> ; off by default
Infrastructure	NODE_TLS_REJECT_UNAUTHORIZED	Not set in production
Client	VITE_API_URL	Optional. Unset, the client derives the API origin from the page host, which is what lets a phone on the same Wi-Fi reach a development backend

DESIGN NOTE Every per-IP ceiling lives in configuration because a security ceiling is also a capacity number: a deployment behind a shared NAT puts every user behind one public address, so a value tuned for a home connection locks out a whole institution. Unset, empty or nonsensical values fall back to the documented default — a typo must never be able to switch rate limiting off.

Appendix D · Source File Index

The complete map of the repository, for the appendices of the written chapters.

Path	Contents
server/index.js	Application assembly: boot checks, middleware order, route mounting, background jobs, error handling
server/blockchain/	crypto.js, ledger.js, engine.js, rules.js, seed.js — 5 modules
server/middleware/	auth.js (protect, adminOnly, session cache), stepUp.js (the step-up gate) — 2 files
server/models/	20 model files: User, Assessment, IntakeRecord, DashboardMetrics, SupplementDetail, UserNotification, ChatThread, ChatMessage, AdminAccount, AdminEvent, Session, MfaTransaction, Passkey, AuthChallenge, BackupCode, PasswordResetToken, SecurityEvent, SubscriptionRequest, SubscriptionCancelRequest, Web3
server/routes/	22 route files plus web3/ with 9: chain, data, ecosystem, govern, guards, index, market, rewards, supply
server/utils/	62 service modules across password, session, MFA, abuse control, entitlement, domain, AI, infrastructure and admin concerns
server/scripts/	check-syntax.js, notify-admins.js, restore-lost-account.js, migrate-pictures-to-disk.js`
server/tools/	patch.js — one-shot verified text replacement
server/Test File/	63 *.test.js suites plus 7 harness/helper files
my-react-app/vite.config.js	Vite + PWA + optimizeDeps contract
my-react-app/src/App.jsx	27 route declarations, 4 guard components, error boundary, session bootstrap, subscription mount, global chat
my-react-app/src/api.js	~110 exported API calls, tab-scoped session storage, shared account directory, lazy WebAuthn SDK
my-react-app/src/api/web3.js	62 provability API calls
my-react-app/src/auth/authState.js	The single reactive identity source of truth
my-react-app/src/hooks/	useAuth, useSubscription, usePlanDay, useNow, useScrolledPast, useSecurityMonitor
my-react-app/src/subscription/	features.js (the entitlement mirror), catalogue.js, paymentCopy.js, sheetActions.js, cancelSheetActions.js
my-react-app/src/Pages/	25 page components plus 19 stylesheets
my-react-app/src/Components/	43 components across navigation, member shell, provability panels, account security, admin console and shared domain
my-react-app/src/utils/	26 modules: wellnessReport, reportTheme, securityReport, exportPDF, recommendationView, adminOverview, slotSchedule, timeSlots, planDay, scoreSync, plan, greeting, passwordPolicy, nameValidation, safeUrl, imageUrl, dataUrl, overlayRegistry, dashboardRefresh, chunkReload, credentialNotice, securityFactor, securityEventLabels, supplementPrefill, dates, sourceAssert
my-react-app/android/	Capacitor project with custom Java: MainActivity.java (210 lines), SafeDownloadName.java (97 lines) and a JVM unit test
my-react-app/public/	manifest.json, favicon.svg, pwa-192x192.png, pwa-512x512.png, apple-touch-icon.png, robots.txt
docs/	USER_MANUAL, USER_GUIDE, DEVELOPER_MANUAL, DEVELOPER_GUIDE, AUTH_SECURITY_REPORT, the patch-notes generator, and this manual's generator
Repository root	README.md (812 lines), SYSTEM_ARCHITECTURE.md, SECURITY.md, SECURITY_AUDIT_REPORT.md, BLOCKCHAIN_FEATURES.md, ADMIN_SETUP.md, TESTING_GUIDE.md, CAPSTONE_CRITIQUE_ANALYSIS.md, and 110 further change documents

Appendix E · Glossary

Term	Definition
argon2id	Memory-hard password hashing function. Preferred over bcrypt because it resists GPU and side-channel attacks; used here via <code>hash-wasm</code> so no native build step is required
Capability / CID	A content address derived from a digest, conventionally written <code>bafy...</code> . Identifies content by its hash rather than by a location
CSP	Content Security Policy. A response header declaring which script, style, frame and connection sources the browser will honour
CSRF	Cross-Site Request Forgery. Mitigated here by the global <code>verifyOrigin</code> guard rather than by cookies alone, since the API is bearer-token based
DID	Decentralized Identifier. Here <code>did:suppliwise:<userId></code> , backed by an ed25519 key pair generated for the member
Entitlement	A server-side permission attached to a subscription tier. The single source of truth is <code>server/utils/entitlements.js</code>
Escalate-only	The priority-flagging invariant: the AI layer may add a clinical flag but can never clear one, so a model failure or hallucination cannot downgrade a rule-based verdict
Free tier / Deluxe / Premium / Ultimate	The four subscription tiers, stored as <code>free / monthly / annual / custom</code> . Only the display names change; the stored ids are the contract
JWT	JSON Web Token. HS256 only, verified with the algorithm pinned; carries the session id so a stateless token still resolves to a revocable session
MFA transaction	A short-lived, attempt-budgeted record issued at the password step that states which second factors are permitted. It is why a leaked second factor alone is not sufficient to sign in
Plan day	The 04:00-to-04:00 window in the member's own timezone, labelled by the date it opens on. Defined once on the server and mirrored on the client
PoW / difficulty	Proof of work. Here, requiring a block hash to begin with three zero characters — roughly 4,096 attempts, sub-millisecond. It demonstrates the mechanism; it is not a security boundary
Priority review	A flag raised on an assessment when the severity engine considers it severe enough to need human review before new assessments proceed
Provenance / anchored	A value that has been committed to the ledger as a transaction with a block index, so it can be re-verified later
RLS	Research Literature Support — the peer-reviewed citation attached to a supplement recommendation, including PMID where available
SSE	Server-Sent Events. A one-way HTTP stream used for live plan changes. Chosen over WebSockets because the traffic is one-directional; it cannot set headers, so it authenticates through a token shim
Step-up	Short-lived re-authentication, traded for password plus current TOTP, required before any security-mutating action. Carried in the <code>X-Step-Up</code> header, never in a URL
Sticky field	A TOTP seed. Cannot be hashed, because the server must run HMAC over it — which is why it is encrypted at rest rather than hashed
Trust proxy	A deployment setting that tells Express to believe <code>X-Forwarded-For</code> . Off by default, because a spoofed value would otherwise poison the IP address used for rate limiting and lockout
Vite optimizer cache	Vite's pre-bundled dependency cache. A package first reached through a dynamic import triggers a re-optimize that invalidates module URLs an open page already holds — which is why <code>optimizeDeps.include</code> completeness is enforced by a test
WebAuthn / passkey	The FIDO2 standard. The member's device creates a key pair; the private key never leaves it, so there is nothing to phish
Workbox	The library behind <code>vite-plugin-pwa</code> that generates the service worker and manages the precache and runtime-caching strategies
Zen-class escalation	*(Not a project term.)* Do not use this; see “escalating lockout ladder” in Section 10.5

Document Control

Field	Value
Document title	SuppliWise — System Manual and Technical Reference
Version	1.0
Status	[Draft / For Review / Final]
Prepared by	[Member 1], [Member 2], [Member 3], [Member 4]
Reviewed by	[Instructor / Adviser]
Institution	[Institution / University Name]
Course	[Course Code and Title]
Academic year	[A.Y. 2026–2027]
Source revision	Branch JDMv6 — 102 commits, 2026-05-27 to 2026-10-02
Generation command	<code>python docs/generate_system_manual.py</code>
Regeneration note	Every figure in this document is derived from the repository. After significant code changes, re-run the generator and re-verify the metrics in Section 2.4.

HOW TO USE THIS MANUAL This document is the consolidated technical baseline for the team's written deliverables. Chapters 1–3 are written to be adapted into the introduction and background chapters. Chapters 4–13 are the system chapters, and each is self-contained enough to be split across individual student submissions — use the module ownership table in Section 14.2 to assign them. The appendices are reference material for the technical appendices. Replace every bracketed placeholder before submission, and re-run the generator whenever the metrics in Section 2.4 change.